Practical 1: Student Information Management System

Objective

To understand Python variables, data types, input/output functions, and formatted printing by developing a simple student information system.

Problem Statement

Develop a Python program that collects the details of a student, including Enrollment Number, Name, Branch, Semester, Date of Birth, Mobile Number, and CGPA. The program should display all information in a neatly formatted report similar to a college student profile.

Functional Requirements

● Accept all details from the user.

● Store data using appropriate variables.

● Display the information using formatted output.

● Calculate whether the student is eligible for placement based on CGPA (for example,

CGPA ≥ 6.5).

Concepts Covered

● Variables

● Data Types

● Input and Output

● String Formatting

● Arithmetic and Comparison Operators

Code:-
```
# Taking student details

enrollment_no = input("Enter Enrollment Number: ")
name = input("Enter Student Name: ")
branch = input("Enter Branch: ")
semester = int(input("Enter Semester: "))
dob = input("Enter Date of Birth (DD-MM-YYYY): ")
mobile = input("Enter Mobile Number: ")
cgpa = float(input("Enter CGPA: "))

# Checking placement eligibility
if cgpa >= 6.5:
    eligibility = "Eligible for Placement"
else:
    eligibility = "Not Eligible for Placement"


print("Enrollment Number :", enrollment_no)
print("Name              :", name)
print("Branch            :", branch)
print("Semester          :", semester)
print("Date of Birth     :", dob)
print("Mobile Number     :", mobile)
print("CGPA              :", cgpa)
print("Placement Status  :", eligibility)
```

Output:-

```
Enter Name: Ayush
Enter Enrollment Number: 240280152035
Enter Branch: AIML
Enter Semester: 5
Enter Date of Birth: 31-12-2006
Enter Mobile Number: 8849741717
Enter CGPA: 8:00

--- Student Information ---
Name: Ayush
Enrollment Number: 240280152035
Branch: AIML
Semester: 5
Date of Birth: 31-12-2006
Mobile Number: 8849741717
CGPA: 8.0
Placement: Eligible
```

Practical 2: Electricity Bill Calculator

Objective

To learn conditional statements by implementing a real-world electricity billing application.

Problem Statement

Develop a Python program that calculates the monthly electricity bill based on the number of units consumed. Different tariff rates should be applied according to predefined slabs. The final bill should also include fixed charges and GST.

Sample Tariff

Units Rate (₹/Unit)

0–100 3.50

101–300 5.00

Above 300 7.50

Fixed Charge = ₹100

GST = 5%

Functional Requirements

- Accept customer name and units consumed.
- Calculate bill according to slab rates.
- Add fixed charges and GST.
- Display a professional electricity bill.

Concepts Covered

- if-elif-else
- Arithmetic Operations
- Variables
- Formatted Output

Code:-

```python
name = input("Enter Customer Name: ")
units = int(input("Enter Units Consumed: "))

if units <= 100:
    bill = units * 3.50
elif units <= 300:
    bill = units * 5.00
else:
    bill = units * 7.50

fixed_charge = 100
subtotal = bill + fixed_charge
gst = subtotal * 5 / 100
total = subtotal + gst

print("\n--- Electricity Bill ---")
print("Customer Name:", name)
print("Units:", units)
print("Energy Charge:", bill)
print("Fixed Charge:", fixed_charge)
print("GST:", gst)
print("Total Bill:", total)
```

Output:-

```
Enter Customer Name: Ayush
Enter Units Consumed: 50

--- Electricity Bill ---
Customer Name: Ayush
Units: 50
Energy Charge: 175.0
Fixed Charge: 100
GST: 13.75
Total Bill: 288.75
```

Practical 3: Student Grade Analyzer

Objective

To understand loops, conditional statements, and mathematical operations using an academic result processing system.

Problem Statement

Develop a program that accepts marks obtained by a student in five subjects. The program should calculate the total marks, percentage, grade, and pass/fail status. Display the complete result in a formatted report card. Grade Policy (Example) Percentage Grade

≥90 A+

80–89 A

70–79 B

60–69 C

50–59 D

Below 50 Fail

Functional Requirements

- Accept marks of five subjects.
- Validate that marks are between 0 and 100.
- Calculate total and percentage.
- Assign grade.
- Display pass/fail status.
- Identify the highest and lowest scoring subjects.

Concepts Covered

- Loops
- Conditional Statements
- Arithmetic Operators
- Variables

Code & Output:-

```python
marks = []

for i in range(5):
    m = int(input("Enter marks: "))
    marks.append(m)

total = sum(marks)
percentage = total / 5

if percentage >= 90:
    grade = "A+"
elif percentage >= 80:
    grade = "A"
elif percentage >= 70:
    grade = "B"
elif percentage >= 60:
    grade = "C"
elif percentage >= 50:
    grade = "D"
else:
    grade = "Fail"

if percentage >= 50:
    result = "Pass"
else:
    result = "Fail"

print("Marks:", marks)
print("Total:", total)
print("Percentage:", percentage)
print("Grade:", grade)
print("Result:", result)

print("Highest Marks:", max(marks))
print("Lowest Marks:", min(marks))
```

```
Enter marks: 90
Enter marks: 100
Enter marks: 47
Enter marks: 67
Enter marks: 56
Marks: [90, 100, 47, 67, 56]
Total: 360
Percentage: 72.0
Grade: B
Result: Pass
Highest Marks: 100
Lowest Marks: 47
```

Practical 4: Inventory Management System using Lists and Dictionaries

Objective

To understand Python's mutable data structures by implementing a simple inventory management application.

Problem Statement

Develop an inventory management system for a small stationery shop. Each product should contain Product ID, Product Name, Quantity Available, and Price. The system should allow users to manage inventory records.

Functional Requirements

The program should provide a menu-driven interface to:

- Add a new product
- Search a product using Product ID
- Update product quantity
- Delete a product
- Display all products
- Calculate total inventory value

Inventory Value = Quantity × Price

Concepts Covered

- Lists
- Dictionaries
- List Operations
- Mutability
- Menu-driven Programming

Code :-

```
products = {}

while True:
    print("\n1. Add")
    print("2. Search")
    print("3. Update")
    print("4. Delete")
    print("5. Display")
    print("6. Total Value")
    print("7. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        id = input("Product ID: ")
        name = input("Product Name: ")
        qty = int(input("Quantity: "))
        price = float(input("Price: "))

        products[id] = [name, qty, price]
        print("Product added")

    elif choice == "2":
        id = input("Product ID: ")
        if id in products:
            print(products[id])
        else:
            print("Not found")

    elif choice == "3":
        id = input("Product ID: ")
        if id in products:
            products[id][1] = int(input("New Quantity: "))
            print("Updated")

    elif choice == "4":
        id = input("Product ID: ")
        if id in products:
            del products[id]
            print("Deleted")

    elif choice == "5":
        print(products)

    elif choice == "6":
        total = 0
        for p in products.values():
            total += p[1] * p[2]
```

```
        print("Total Value:", total)

    elif choice == "7":
        break

    else:
        print("Invalid choice")
```

Output:-

```
1. Add
2. Search
3. Update
4. Delete
5. Display
6. Total Value
7. Exit
Enter choice: 1
Product ID: P101
Product Name: pen
Quantity: 10
Price: 5
Product added
```

Practical 5: Online Shopping Cart Simulation

Objective

To understand lists, dictionaries, loops, and decision-making by simulating an online shopping cart.

Problem Statement

Develop a shopping cart application similar to an e-commerce website. Users should be able to browse available products, add items to their cart, remove items, and generate the final invoice. Functional Requirements

● Display available products.

● Add products to the cart.

● Remove products from the cart.

● Modify quantities.

● Calculate subtotal.

● Apply discount:

○ 10% discount for purchases above ₹5000.

● Calculate GST (18%).

● Generate the final bill.

Sample Output

Product Qty Price

Laptop 1 45000

Mouse 2 1200

Subtotal ₹47400

Discount ₹4740

GST ₹7680

Total Amount ₹50340

Concepts Covered

- Lists
- Dictionaries
- Loops
- Strings
- Conditional Statements

Code:-

```
products = {
    "Laptop": 45000,
    "Mouse": 1200,
    "Keyboard": 2000
}

cart = {}

while True:
    print("\n1. Show Products")
    print("2. Add Product")
    print("3. Remove Product")
    print("4. Change Quantity")
    print("5. Bill")
    print("6. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        for name, price in products.items():
            print(name, "₹", price)

    elif choice == "2":
        name = input("Enter product name: ")
        if name in products:
            qty = int(input("Enter quantity: "))
            cart[name] = qty
            print("Added to cart")
        else:
            print("Product not found")

    elif choice == "3":
        name = input("Enter product name: ")
        if name in cart:
            del cart[name]
```

```
            print("Removed")

    elif choice == "4":
        name = input("Enter product name: ")
        if name in cart:
            cart[name] = int(input("Enter new quantity: "))
            print("Quantity updated")

    elif choice == "5":
        subtotal = 0

        for name, qty in cart.items():
            subtotal += products[name] * qty

        if subtotal > 5000:
            discount = subtotal * 10 / 100
        else:
            discount = 0

        gst = (subtotal - discount) * 18 / 100
        total = subtotal - discount + gst

        print("\n--- Final Bill ---")
        print("Subtotal:", subtotal)
        print("Discount:", discount)
        print("GST:", gst)
        print("Total Amount:", total)

    elif choice == "6":
        break

    else:
        print("Invalid choice")
```

Output:-

```
Enter choice: 1
Laptop ₹ 45000
Mouse ₹ 1200
Keyboard ₹ 2000

1. Show Products
2. Add Product
3. Remove Product
4. Change Quantity
5. Bill
6. Exit
Enter choice: 2
Enter product name: Mouse
Enter quantity: 4
Added to cart

1. Show Products
2. Add Product
3. Remove Product
4. Change Quantity
5. Bill
6. Exit
Enter choice: 5

--- Final Bill ---
Subtotal: 4800
Discount: 0
GST: 864.0
Total Amount: 5664.0
```

Practical 6: Password Strength Validator

Objective

To understand string manipulation and conditional logic by implementing a password validation system.

Problem Statement

Develop a Python program that checks whether a password satisfies basic cybersecurity requirements before allowing a user to register.

Password Policy

A valid password must contain:

- Minimum 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one digit
- At least one special character (@, #, $, %, &, *)

Functional Requirements

- Accept password from the user.
- Check each validation rule.
- Display specific messages indicating which criteria are not satisfied.
- Display "Strong Password" only if all conditions are met.

Example Output

Enter Password:

Python@123

Password Strength: Strong

or

Enter Password:

python123

Missing:

✓ Uppercase Letter

✓ Special Character

Password Strength: Weak

Concepts Covered

- Strings
- Loops
- Conditional Statements
- String Functions

Code:-

```
password = input("Enter Password: ")

upper = False
lower = False
digit = False
special = False

for ch in password:
    if ch.isupper():
        upper = True
    elif ch.islower():
        lower = True
    elif ch.isdigit():
        digit = True
    elif ch in "@#$%&*":
        special = True

if len(password) < 8:
    print("Missing: Minimum 8 characters")

if not upper:
    print("Missing: Uppercase Letter")

if not lower:
    print("Missing: Lowercase Letter")

if not digit:
    print("Missing: Digit")
```

```
if not special:
    print("Missing: Special Character")

if len(password) >= 8 and upper and lower and digit and special:
    print("Password Strength: Strong")
else:
    print("Password Strength: Weak")
```

Output:-

```
Enter Password: acb123@#
Missing: Uppercase Letter
Password Strength: Weak
(.venv) PS C:\Users\Dell\OneDr
n.exe c:/Users/Dell/OneDrive/D
Enter Password: deep@#123
Missing: Uppercase Letter
Password Strength: Weak
```

Unit 2: Functions, Modules and Files

Practical 1: Library Management using Functions

Description:
Develop a menu-driven library management system using user-defined functions for issuing
books, returning books, searching books, and displaying available books.

Concepts Covered
- Functions
- Arguments
- Return Values
- Modular Programming

Code:-

```
books = [
    "Python Programming",
    "Data Structures",
    "Machine Learning",
    "Artificial Intelligence",
    "Operating System",
    "Database Management System"
]

issued_books = []

def display_books():

    if len(books) == 0:
        print("\nNo books are available in the library.")
    else:
        print("\nAvailable Books:")

        for i, book in enumerate(books, start=1):
            print(i, ".", book)

def issue_book():
```

```
    if len(books) == 0:
        print("\nNo books are available to issue.")
        return

    display_books()

    book_name = input("\nEnter the name of the book to issue: ")

    if book_name in books:
        books.remove(book_name)
        issued_books.append(book_name)

        print("\nBook issued successfully.")

    else:
        print("\nBook not found.")


def return_book():

    if len(issued_books) == 0:
        print("\nNo books have been issued.")
        return

    print("\nIssued Books:")

    for book in issued_books:
        print("-", book)

    book_name = input("\nEnter the name of the book to return: ")

    if book_name in issued_books:
        issued_books.remove(book_name)
        books.append(book_name)

        print("\nBook returned successfully.")

    else:
        print("\nThis book was not issued.")

def search_book():

    book_name = input("\nEnter the name of the book to search: ")

    if book_name in books:
        print("\nBook is available in the library.")

    elif book_name in issued_books:
        print("\nBook has already been issued.")

    else:
        print("\nBook not found.")


def menu():
```

```
    while True:

        print("\n========== LIBRARY MANAGEMENT SYSTEM ==========")

        print("1. Display Available Books")
        print("2. Issue Book")
        print("3. Return Book")
        print("4. Search Book")
        print("5. Exit")

        choice = input("\nEnter your choice: ")

        if choice == "1":
            display_books()

        elif choice == "2":
            issue_book()

        elif choice == "3":
            return_book()

        elif choice == "4":
            search_book()

        elif choice == "5":
            print("\nThank you for using the Library Management
System.")
            break

        else:
            print("\nInvalid choice. Please try again.")


menu()
```

Output:-

```
========== LIBRARY MANAGEMENT SYSTEM ==========
1. Display Available Books
2. Issue Book
3. Return Book
4. Search Book
5. Exit

Enter your choice: 1

Available Books:
1 . Python Programming
2 . Data Structures
3 . Machine Learning
4 . Artificial Intelligence
5 . Operating System
6 . Database Management System
```

Practical 2: Payroll Processing System

Description:
Create a payroll system where salary, HRA, DA, PF, tax deduction, and net salary are
calculated using separate functions.

Concepts Covered
- Functions
- Default Arguments
- Multiple Return Values

Code:-

```
def calculate_hra(basic_salary, hra_percentage=20):
    return (basic_salary * hra_percentage) / 100


def calculate_da(basic_salary, da_percentage=10):
    return (basic_salary * da_percentage) / 100

def calculate_pf(basic_salary, pf_percentage=12):
    return (basic_salary * pf_percentage) / 100

def calculate_tax(gross_salary, tax_percentage=5):
    return (gross_salary * tax_percentage) / 100


def calculate_salary(basic_salary):

    hra = calculate_hra(basic_salary)

    da = calculate_da(basic_salary)

    pf = calculate_pf(basic_salary)

    gross_salary = basic_salary + hra + da

    tax = calculate_tax(gross_salary)

    net_salary = gross_salary - pf - tax

    return hra, da, pf, tax, gross_salary, net_salary

basic_salary = float(input("Enter Basic Salary: "))

hra, da, pf, tax, gross_salary, net_salary = calculate_salary(
    basic_salary
```

```
)

print("\n PAYROLL DETAILS ")

print("Basic Salary:", basic_salary)

print("HRA:", hra)

print("DA:", da)

print("PF:", pf)

print("Tax Deduction:", tax)

print("Gross Salary:", gross_salary)

print("Net Salary:", net_salary)
```

Output:-

```
 PAYROLL DETAILS
Basic Salary: 50000.0
HRA: 10000.0
DA: 5000.0
PF: 6000.0
Tax Deduction: 3250.0
Gross Salary: 65000.0
Net Salary: 55750.0
```

Practical 3: Student Result Analysis using Lambda Functions

Description:

Store student marks and use lambda functions with map(), filter(), and reduce() to calculate
grades, identify toppers, and compute class averages.

Concepts Covered

- Lambda Functions
- map()
- filter()
- reduce()

Code:-

```
from functools import reduce

students = {
    "Raj": 95,
    "Ayush": 92,
    "Vidhi": 96,
    "Yashvi": 68,
    "Het": 88,
    "Meet": 55
}

def calculate_grade(mark):

    if mark >= 90:
        return "A+"

    elif mark >= 80:
        return "A"

    elif mark >= 70:
        return "B"

    elif mark >= 60:
        return "C"

    else:
        return "D"

grades = list(
    map(
        lambda item: (
            item[0],
            item[1],
            calculate_grade(item[1])
        ),
        students.items()
```

```
    )
)

toppers = list(
    filter(
        lambda item: item[1] >= 85,
        students.items()
    )
)


total_marks = reduce(
    lambda x, y: x + y,
    students.values()
)


average_marks = total_marks / len(students)


print("\nSTUDENT DETAILS ")

for name, marks, grade in grades:
    print(name, "-", marks, "-", grade)

print("\n TOPPERS ")

for name, marks in toppers:
    print(name, "-", marks)


print("\nClass Average:", average_marks)
```

Output:-

```
STUDENT DETAILS
Raj - 95 - A+
Ayush - 92 - A+
Vidhi - 96 - A+
Yashvi - 68 - C
Het - 88 - A
Meet - 55 - D

 TOPPERS
Raj - 95
Ayush - 92
Vidhi - 96
Het - 88
```

Practical 4: Employee Attendance Module

Description:
Create your own Python module to calculate employee attendance percentage, working days,
leaves, and overtime.

Concepts Covered
- Modules
- Import Statement
- Functions

Code:-

```
# attendance_module.py

def attendance_percentage(total_days, present_days):

    percentage = (present_days / total_days) * 100

    return percentage


def working_days(total_days, leave_days):

    return total_days - leave_days

def calculate_leaves(total_days, present_days):

    return total_days - present_days

def calculate_overtime(extra_hours):

    return extra_hours

# main.py


import attendance_module as am


total_days = int(input("Enter total working days: "))

present_days = int(input("Enter present days: "))

overtime_hours = float(input("Enter overtime hours: "))


leave_days = am.calculate_leaves(
    total_days,
    present_days
```

```
)

attendance = am.attendance_percentage(
    total_days,
    present_days
)

working = am.working_days(
    total_days,
    leave_days
)

overtime = am.calculate_overtime(
    overtime_hours
)


print("\nEMPLOYEE ATTENDANCE REPORT")

print("Total Working Days:", total_days)

print("Present Days:", present_days)

print("Leaves:", leave_days)

print("Attendance Percentage:", attendance, "%")

print("Working Days:", working)

print("Overtime Hours:", overtime)
```

Output:-

```
Enter total working days: 250
Enter present days: 237
Enter overtime hours: 15

EMPLOYEE ATTENDANCE REPORT
Total Working Days: 250
Present Days: 237
Leaves: 13
Attendance Percentage: 94.8 %
Working Days: 237
Overtime Hours: 15.0
```

Practical 5: Electricity Consumption Report Generator

Description:
Develop a program that reads monthly electricity consumption from a file, processes the data
using functions, and generates a summary report.

Concepts Covered
- Functions
- Files
- Modules

Code:-

```
# electricity_module.py

def read_data(filename):

    consumption = []

    file = open(filename, "r")

    for line in file:
        consumption.append(float(line.strip()))

    file.close()

    return consumption


def total_consumption(data):

    return sum(data)


def average_consumption(data):

    return sum(data) / len(data)


def generate_report(data):

    total = total_consumption(data)

    average = average_consumption(data)

    print("\nELECTRICITY CONSUMPTION REPORT")

    print("Monthly Consumption (Units):", data)

    print("Total Consumption:", total, "Units")
```

```
    print("Average Consumption:", average, "Units")
```

electricity_data.txt

```
320
280
350
300
400
360
290
330
310
370
340
380
```

```
# main.py

import electricity_module as em

data = em.read_data("/Users/ladanirajmukeshbhai/PYTHON/PP_Unit-1_Prac-5/electricity_data.txt")

em.generate_report(data)
```

Output:-

```
ELECTRICITY CONSUMPTION REPORT
Monthly Consumption (Units): [320.0, 280.0, 350.0, 300.0, 400.0, 360.0, 290.0, 330.0, 310.0, 370.0, 340.0, 380.0]
Total Consumption: 4030.0 Units
Average Consumption: 335.8333333333333 Units
```

Name:- Ayush Lakum Enrolment No:- 240280152035

Practical 6: Bank Interest Calculator using Recursive Functions

Description:
Implement recursive functions to calculate compound interest over multiple years and compare
it with simple interest.

Concepts Covered
- Recursion
- Functions

Code:-

```
def compound_interest(principal, rate, years):

    if years == 0:
        return principal

    return compound_interest(
        principal * (1 + rate / 100),
        rate,
        years - 1
    )


def simple_interest(principal, rate, years):

    return principal + (principal * rate * years) / 100


principal = float(input("Enter the principal amount: "))

rate = float(input("Enter the annual interest rate (%): "))

years = int(input("Enter the number of years: "))


compound_amount = compound_interest(
    principal,
    rate,
    years
)

simple_amount = simple_interest(
    principal,
    rate,
    years
)
```

```
print("\nINTEREST REPORT ")

print("Principal Amount:", principal)

print("Rate of Interest:", rate, "%")

print("Number of Years:", years)

print("Compound Interest Amount:", round(compound_amount, 2))

print("Simple Interest Amount:", round(simple_amount, 2))

print(
    "Difference:",
    round(compound_amount - simple_amount, 2)
)
```

Output:-

```
INTEREST REPORT
Principal Amount: 4000000.0
Rate of Interest: 2.0 %
Number of Years: 5
Compound Interest Amount: 4416323.21
Simple Interest Amount: 4400000.0
Difference: 16323.21
```

Unit 3: Exception Handling and File

Handling

Practical 1: ATM Transaction System

Description:

Develop an ATM simulation that handles invalid PINs, insufficient balance, invalid withdrawal

amounts, and transaction failures using exception handling.

Concepts Covered

- try
- except
- finally
- User-defined Exceptions

```python
# Practical 1: ATM Transaction System

class InvalidPINException(Exception):
    pass


class InsufficientBalanceException(Exception):
    pass


class InvalidAmountException(Exception):
    pass


class TransactionFailedException(Exception):
    pass


# ATM details
correct_pin = "1234"
balance = 10000


try:
    # PIN verification
    pin = input("Enter your PIN: ")

    if pin != correct_pin:
        raise InvalidPINException("Invalid PIN!")

    print("\nLogin Successful!")
    print("Current Balance: ₹", balance)

    # Withdrawal amount
    amount = float(input("Enter withdrawal amount: ₹"))

    # Check amount
    if amount <= 0:
        raise InvalidAmountException(
            "Withdrawal amount must be greater than 0."
        )

    if amount % 100 != 0:
        raise InvalidAmountException(
            "Please enter amount in multiples of ₹100."
        )

    # Check balance
    if amount > balance:
        raise InsufficientBalanceException(
            "Insufficient balance!"
        )

    # Transaction
    balance -= amount

    if balance < 0:
        raise TransactionFailedException(
            "Transaction failed!"
        )

    print("\nTransaction Successful!")
    print("Amount Withdrawn: ₹", amount)
    print("Remaining Balance: ₹", balance)

except InvalidPINException as e:
    print("Error:", e)

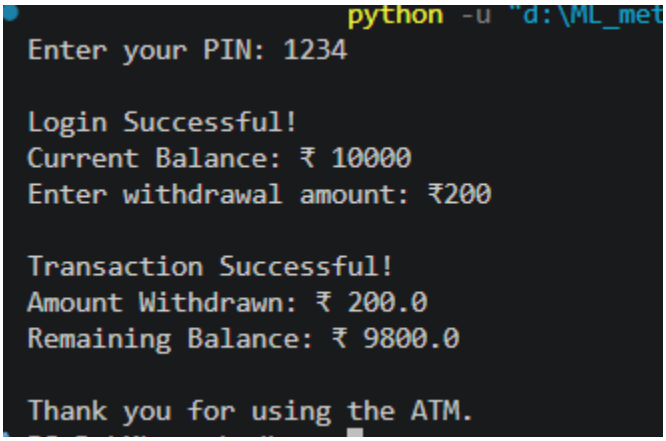
except InvalidAmountException as e:
    print("Error:", e)

except InsufficientBalanceException as e:
    print("Error:", e)

except TransactionFailedException as e:
    print("Error:", e)

except Exception as e:
    print("Unexpected Error:", e)

finally:
    print("\nThank you for using the ATM.")
```



Practical 2: Student Report Card File System

Description:

Store student records in a text file and provide options to add, search, modify, and display records.

Concepts Covered

- Text Files
- File Handling
- with Statement

Input:

FILE_NAME = "students.txt"

```
# Add Student
def add_student():
    roll_no = input("Enter Roll Number: ")
    name = input("Enter Student Name: ")
    marks = input("Enter Marks: ")
```

```
    with open(FILE_NAME, "a") as file:
        file.write(roll_no + "," + name + "," + marks + "\n")

    print("Student record added successfully!")

# Display Students
def display_students():
    try:
        with open(FILE_NAME, "r") as file:
            records = file.readlines()

            if not records:
                print("No student records found.")
                return

            print("\n--- Student Records ---")

            for record in records:
                roll, name, marks = record.strip().split(",")
                print("Roll No :", roll)
                print("Name    :", name)
                print("Marks   :", marks)
                print("-----------------------")

    except FileNotFoundError:
```

```
        print("No student records found.")


# Search Student
def search_student():
    roll_no = input("Enter Roll Number to search: ")

    try:
        with open(FILE_NAME, "r") as file:
            found = False

            for record in file:
                roll, name, marks = record.strip().split(",")

                if roll == roll_no:
                    print("\nStudent Found!")
                    print("Roll No :", roll)
                    print("Name    :", name)
                    print("Marks   :", marks)
                    found = True
                    break

            if not found:
                print("Student record not found.")

    except FileNotFoundError:
        print("No student records found.")
```

```
# Modify Student
def modify_student():
    roll_no = input("Enter Roll Number to modify: ")

    try:
        with open(FILE_NAME, "r") as file:
            records = file.readlines()

        found = False

        with open(FILE_NAME, "w") as file:
            for record in records:
                roll, name, marks = record.strip().split(",")

                if roll == roll_no:
                    new_name = input("Enter New Name: ")
                    new_marks = input("Enter New Marks: ")

                    file.write(
                        roll + "," + new_name + "," + new_marks + "\n"
                    )

                    found = True
                else:
                    file.write(record)
```

```
        if found:
            print("Student record modified successfully!")
        else:
            print("Student record not found.")

    except FileNotFoundError:
        print("No student records found.")

# Main Menu
while True:
    print("\n===== Student Report Card System =====")
    print("1. Add Student")
    print("2. Search Student")
    print("3. Modify Student")
    print("4. Display All Students")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        add_student()

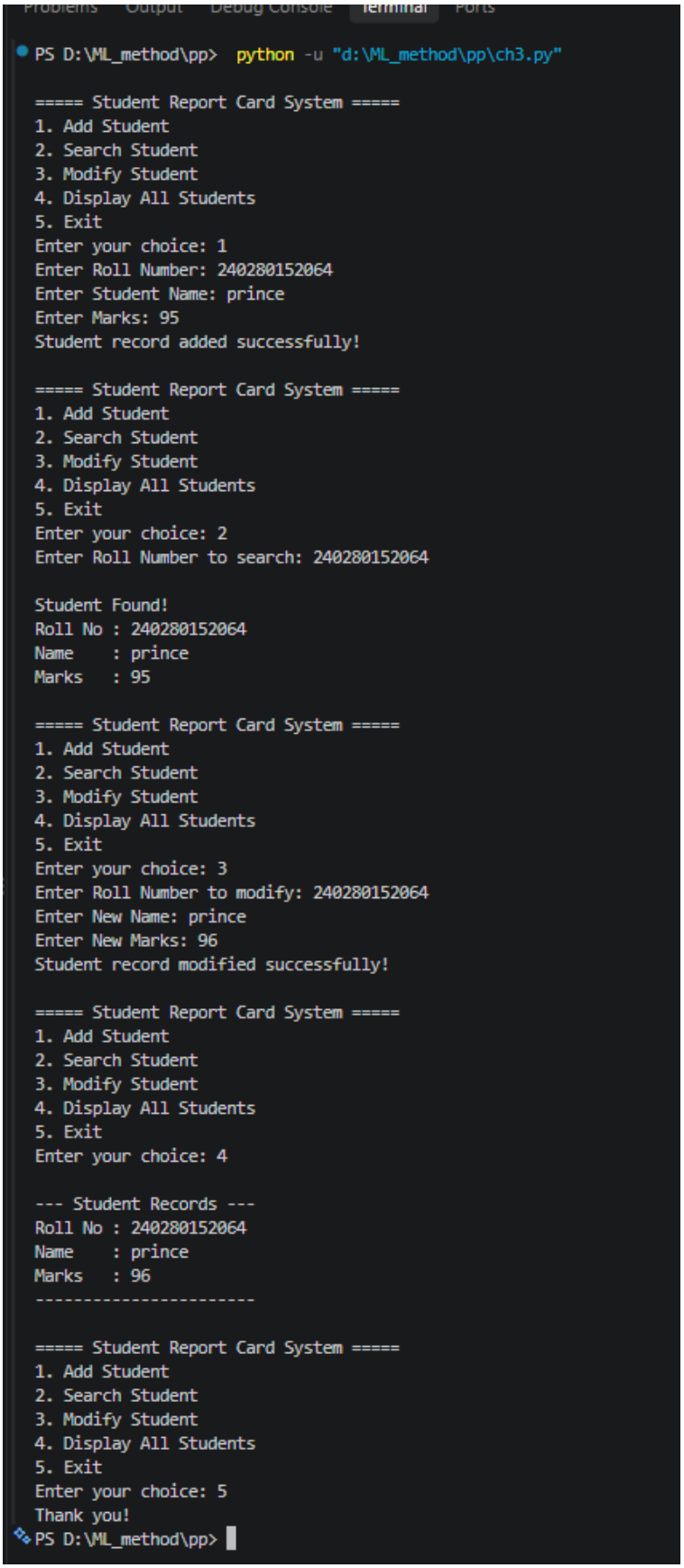
    elif choice == "2":
        search_student()
```

```
elif choice == "3":
    modify_student()

elif choice == "4":
    display_students()

elif choice == "5":
    print("Thank you!")
    break

else:
    print("Invalid choice!")
```


Problems Output Debug Console Terminal Ports
PS D:\ML_method\pp> python -u "d:\ML_method\pp\ch3.py"
===== Student Report Card System =====
1. Add Student
2. Search Student
3. Modify Student
4. Display All Students
5. Exit
Enter your choice: 1
Enter Roll Number: 240280152064
Enter Student Name: prince
Enter Marks: 95
Student record added successfully!
===== Student Report Card System =====
1. Add Student
2. Search Student
3. Modify Student
4. Display All Students
5. Exit
Enter your choice: 2
Enter Roll Number to search: 240280152064
Student Found!
Roll No : 240280152064
Name : prince
Marks : 95
===== Student Report Card System =====
1. Add Student
2. Search Student
3. Modify Student
4. Display All Students
5. Exit
Enter your choice: 3
Enter Roll Number to modify: 240280152064
Enter New Name: prince
Enter New Marks: 96
Student record modified successfully!
===== Student Report Card System =====
1. Add Student
2. Search Student
3. Modify Student
4. Display All Students
5. Exit
Enter your choice: 4
--- Student Records ---
Roll No : 240280152064
Name : prince
Marks : 96

===== Student Report Card System =====
1. Add Student
2. Search Student
3. Modify Student
4. Display All Students
5. Exit
Enter your choice: 5
Thank you!
PS D:\ML_method\pp>

Practical 3: Hospital Patient Record System

Description:

Maintain patient records using binary files and Python Pickle for permanent storage.

Concepts Covered

● Pickle

● Binary Files

Input:

```
import pickle


FILE_NAME = "patients.dat"



def add_patient():
    patient = {
        "id": input("Enter Patient ID: "),
        "name": input("Enter Patient Name: "),
        "age": int(input("Enter Age: ")),
        "disease": input("Enter Disease: "),
        "doctor": input("Enter Doctor Name: ")
    }

    with open(FILE_NAME, "ab") as file:
        pickle.dump(patient, file)

    print("Patient record added successfully.")
```

```
def display_patients():
    try:
        with open(FILE_NAME, "rb") as file:
            print("\n--- Patient Records ---")

            while True:
                try:
                    patient = pickle.load(file)

                    print("Patient ID :", patient["id"])
                    print("Name      :", patient["name"])
                    print("Age       :", patient["age"])
                    print("Disease   :", patient["disease"])
                    print("Doctor    :", patient["doctor"])
                    print("-----------------------")

                except EOFError:
                    break

    except FileNotFoundError:
        print("No patient records found.")


def search_patient():
```

```
search_id = input("Enter Patient ID to search: ")

try:
    with open(FILE_NAME, "rb") as file:
        while True:
            try:
                patient = pickle.load(file)

                if patient["id"] == search_id:
                    print("\nPatient Found!")
                    print("Patient ID :", patient["id"])
                    print("Name      :", patient["name"])
                    print("Age       :", patient["age"])
                    print("Disease   :", patient["disease"])
                    print("Doctor    :", patient["doctor"])
                    return

            except EOFError:
                break

    print("Patient not found.")

except FileNotFoundError:
    print("No patient records found.")
```

```
def main():
    while True:
        print("\n===== Hospital Patient Record System =====")
        print("1. Add Patient")
        print("2. Display All Patients")
        print("3. Search Patient")
        print("4. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            add_patient()

        elif choice == "2":
            display_patients()

        elif choice == "3":
            search_patient()

        elif choice == "4":
            print("Thank you!")
            break

        else:
            print("Invalid choice!")
```

main()

```
PS D:\ML_method\pp> python -u "d:\ML_method\pp\ch3.py"

===== Hospital Patient Record System =====
1. Add Patient
2. Display All Patients
3. Search Patient
4. Exit
Enter your choice: 1
Enter Patient ID: 1
Enter Patient Name: Ayush
Enter Age: 23
Enter Disease: hiv
Enter Doctor Name: priyansh
Patient record added successfully.

===== Hospital Patient Record System =====
1. Add Patient
2. Display All Patients
3. Search Patient
4. Exit
Enter your choice: 2

--- Patient Records ---
Patient ID : 12
Name       Ayush
Age        : 23
Disease    : hiv
Doctor     : priyansh
----------------------

===== Hospital Patient Record System =====
1. Add Patient
2. Display All Patients
3. Search Patient
4. Exit
Enter your choice: 3
Enter Patient ID to search: 12

Patient Found!
Patient ID : 12
Name       Ayush
Age        : 23
Disease    : hiv
Doctor     : priyansh

===== Hospital Patient Record System =====
1. Add Patient
2. Display All Patients
3. Search Patient
4. Exit
Enter your choice: 4
Thank you!
PS D:\ML_method\pp>
```

Practical 4: Log File Analyzer

Description:

Read a system log file, count different types of errors, and generate a summary report while logging exceptions.

Concepts Covered

● File Handling

● Logging

● Exception Handling

Input:

```
import logging
# Configure logging
logging.basicConfig(
  filename="error_log.txt",
  level=logging.ERROR,
  format="%(asctime)s - %(levelname)s - %(message)s"
)
LOG_FILE = "system.log"
REPORT_FILE = "summary_report.txt"
def analyze_log():
  error_count = {}
  try:
    with open(LOG_FILE, "r") as file:

      for line in file:
        line = line.strip()
```

```
            # Check for error lines
            if "ERROR" in line:
                # Get error type
                parts = line.split("ERROR:")
                if len(parts) > 1:
                    error_type = parts[1].strip()
                    # Count error
                    error_count[error_type] = \
                        error_count.get(error_type, 0) + 1
    # Generate summary report
    with open(REPORT_FILE, "w") as report:
        report.write("===== LOG FILE SUMMARY =====\n\n")
        if error_count:
            for error, count in error_count.items():
                report.write(f"{error}: {count}\n")
        else:
            report.write("No errors found.\n")
    print("Log analysis completed successfully.")
    print("Summary report created:", REPORT_FILE)
    print("\n===== Error Summary =====")
    for error, count in error_count.items():
        print(f"{error}: {count}")
except FileNotFoundError:
    logging.error("System log file not found.")
    print("Error: System log file not found.")
except PermissionError:
```

```
        logging.error("Permission denied while accessing the log file.")
        print("Error: Permission denied.")
    except Exception as e:
        logging.error(f"Unexpected error: {e}")
        print("An unexpected error occurred.")
analyze_log()
```

file

2026-10-01 INFO: System started

2026-10-01 ERROR: File Not Found

2026-10-01 INFO: User logged in

2026-10-01 ERROR: Database Connection Failed

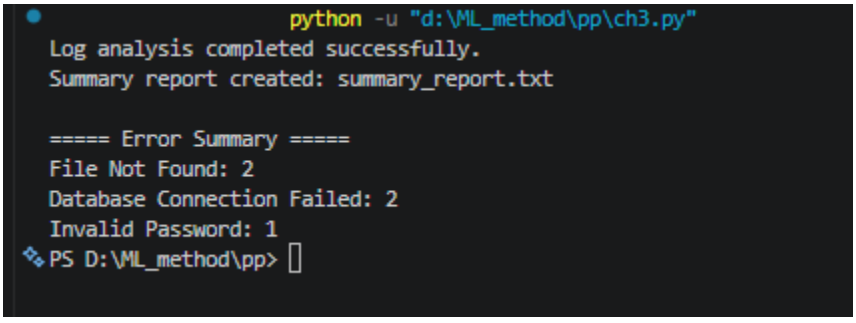
2026-10-01 ERROR: File Not Found

2026-10-01 WARNING: Low Memory

2026-10-01 ERROR: Invalid Password

2026-10-01 ERROR: Database Connection Failed

Output:



Practical 5: Employee Salary Management using Binary Files

Description:

Store employee records in binary format and provide searching, updating, and deletion facilities.

Concepts Covered

- Binary Files
- seek()
- tell()

Input:

```
import pickle
import os

FILE_NAME = "employees.dat"



def add_employee():
  employee = {
    "id": int(input("Enter Employee ID: ")),
    "name": input("Enter Employee Name: "),
    "salary": float(input("Enter Salary: ")),
    "department": input("Enter Department: ")
  }

  with open(FILE_NAME, "ab") as file:
    pickle.dump(employee, file)
```

```
    print("Employee added successfully.")



def display_employees():
    try:
        with open(FILE_NAME, "rb") as file:

            print("\n===== Employee Records =====")

            while True:
                try:
                    employee = pickle.load(file)

                    print("ID       :", employee["id"])
                    print("Name     :", employee["name"])
                    print("Salary   :", employee["salary"])
                    print("Department :", employee["department"])
                    print("----------------------------")

                except EOFError:
                    break

    except FileNotFoundError:
        print("No employee records found.")
```

```
def search_employee():
    emp_id = int(input("Enter Employee ID to search: "))

    try:
        with open(FILE_NAME, "rb") as file:

            while True:
                try:
                    employee = pickle.load(file)

                    if employee["id"] == emp_id:
                        print("\nEmployee Found!")
                        print("ID       :", employee["id"])
                        print("Name     :", employee["name"])
                        print("Salary   :", employee["salary"])
                        print("Department :", employee["department"])
                        return

                except EOFError:
                    break

        print("Employee not found.")

    except FileNotFoundError:
        print("No employee records found.")
```

```
def update_employee():
    emp_id = int(input("Enter Employee ID to update: "))

    try:
        with open(FILE_NAME, "rb") as file:
            employees = []

            while True:
                try:
                    position = file.tell()
                    employee = pickle.load(file)

                    if employee["id"] == emp_id:
                        print("\nEmployee found at position:", position)

                        employee["name"] = input("Enter new name: ")
                        employee["salary"] = float(
                            input("Enter new salary: ")
                        )
                        employee["department"] = input(
                            "Enter new department: "
                        )

                    employees.append(employee)
```

```
            except EOFError:
                break

        with open(FILE_NAME, "wb") as file:
            for employee in employees:
                pickle.dump(employee, file)

        print("Employee updated successfully.")

    except FileNotFoundError:
        print("No employee records found.")



def delete_employee():
    emp_id = int(input("Enter Employee ID to delete: "))

    try:
        with open(FILE_NAME, "rb") as file:
            employees = []
            found = False

            while True:
                try:
                    employee = pickle.load(file)
```

```
            if employee["id"] == emp_id:
                found = True
                continue

            employees.append(employee)

        except EOFError:
            break

    if found:
        with open(FILE_NAME, "wb") as file:
            for employee in employees:
                pickle.dump(employee, file)

        print("Employee deleted successfully.")
    else:
        print("Employee not found.")

except FileNotFoundError:
    print("No employee records found.")

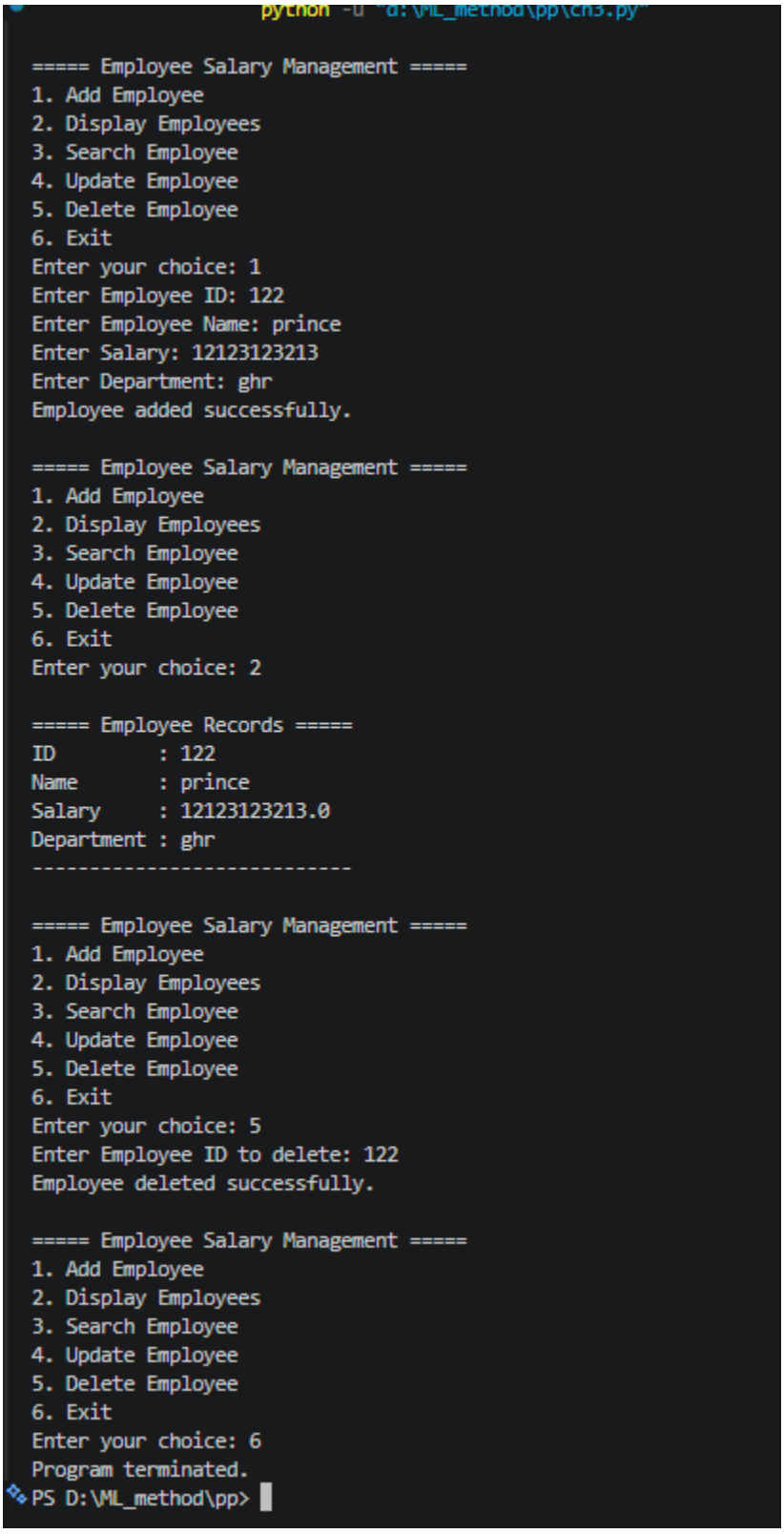

def main():
    while True:

        print("\n===== Employee Salary Management =====")
```

```
    print("1. Add Employee")
    print("2. Display Employees")
    print("3. Search Employee")
    print("4. Update Employee")
    print("5. Delete Employee")
    print("6. Exit")

    choice = input("Enter your choice: ")
    if choice == "1":
      add_employee()
    elif choice == "2":
      display_employees()
    elif choice == "3":
      search_employee()
    elif choice == "4":
      update_employee()
    elif choice == "5":
      delete_employee()
    elif choice == "6":
      print("Program terminated.")
      break
    else:
      print("Invalid choice!")
main()
```

output:


python -u "d:\ML_method\pp\ch3.py"
===== Employee Salary Management =====
1. Add Employee
2. Display Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Exit
Enter your choice: 1
Enter Employee ID: 122
Enter Employee Name: prince
Enter Salary: 12123123213
Enter Department: ghr
Employee added successfully.
===== Employee Salary Management =====
1. Add Employee
2. Display Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Exit
Enter your choice: 2
===== Employee Records =====
ID : 122
Name : prince
Salary : 12123123213.0
Department : ghr

===== Employee Salary Management =====
1. Add Employee
2. Display Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Exit
Enter your choice: 5
Enter Employee ID to delete: 122
Employee deleted successfully.
===== Employee Salary Management =====
1. Add Employee
2. Display Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Exit
Enter your choice: 6
Program terminated.
PS D:\ML_method\pp>

Practical 6: Online Examination Result Backup System

Description:

Create a backup utility that stores examination results into binary files and restores them whenever required.

Concepts Covered

● Pickle

● File Handling

Input:

```
import pickle
import os

RESULT_FILE = "exam_results.dat"
BACKUP_FILE = "exam_backup.dat"



def add_result():
  result = {
    "roll_no": int(input("Enter Roll Number: ")),
    "name": input("Enter Student Name: "),
    "subject": input("Enter Subject: "),
    "marks": float(input("Enter Marks: "))
  }

  with open(RESULT_FILE, "ab") as file:
    pickle.dump(result, file)
```

```
    print("Result added successfully.")



def display_results():
    try:
        with open(RESULT_FILE, "rb") as file:

            print("\n===== Examination Results =====")

            while True:
                try:
                    result = pickle.load(file)

                    print("Roll No :", result["roll_no"])
                    print("Name    :", result["name"])
                    print("Subject :", result["subject"])
                    print("Marks   :", result["marks"])
                    print("------------------------------")

                except EOFError:
                    break
    except FileNotFoundError:
        print("No result file found.")



def create_backup():
```

```
    try:
        with open(RESULT_FILE, "rb") as source:
            with open(BACKUP_FILE, "wb") as backup:


                while True:
                    try:
                        result = pickle.load(source)
                        pickle.dump(result, backup)
                    except EOFError:
                        break
        print("Backup created successfully.")
    except FileNotFoundError:
        print("No examination results found.")
def restore_backup():
    try:
        with open(BACKUP_FILE, "rb") as backup:
            with open(RESULT_FILE, "wb") as result_file:
                while True:
                    try:
                        result = pickle.load(backup)
                        pickle.dump(result, result_file)
                    except EOFError:
                        break
        print("Results restored successfully.")
    except FileNotFoundError:
        print("Backup file not found.")
```

```
def main():
    while True:
        print("\n===== Online Examination Result System =====")
        print("1. Add Result")
        print("2. Display Results")
        print("3. Create Backup")
        print("4. Restore Backup")
        print("5. Exit")
        choice = input("Enter your choice: ")
        if choice == "1":
            add_result()
        elif choice == "2":
            display_results()
        elif choice == "3":
            create_backup()
        elif choice == "4":
            restore_backup()
        elif choice == "5":
            print("Program terminated.")
            break

        else:
            print("Invalid choice!")



main()
```

output:

```
===== Online Examination Result System =====
1. Add Result
2. Display Results
3. Create Backup
4. Restore Backup
5. Exit
Enter your choice: 1
Enter Roll Number: 11111111111
Enter Student Name: prince
Enter Subject: pp
Enter Marks: 100
Result added successfully.

===== Online Examination Result System =====
1. Add Result
2. Display Results
3. Create Backup
4. Restore Backup
5. Exit
Enter your choice: 2

===== Examination Results =====
Roll No : 11111111111
Name    : prince
Subject : pp
Marks   : 100.0
-----------------------------

===== Online Examination Result System =====
1. Add Result
2. Display Results
3. Create Backup
4. Restore Backup
5. Exit
Enter your choice: 3
Backup created successfully.

===== Online Examination Result System =====
1. Add Result
2. Display Results
3. Create Backup
4. Restore Backup
5. Exit
Enter your choice:
```

Unit 4: Classes and Object-Oriented

Programming

Practical 1: Banking Management System

Description:

Develop a banking application that allows account creation, deposits, withdrawals, balance

inquiry, and transaction history using classes and objects.

Concepts Covered

- Classes
- Objects
- Constructors

Input:

```
class BankAccount:

  # Constructor
  def __init__(self, account_no, name, balance=0):
    self.account_no = account_no
    self.name = name
    self.balance = balance
    self.transactions = []

  # Deposit money
```

```
def deposit(self, amount):
    if amount > 0:
        self.balance += amount
        self.transactions.append(f"Deposited: Rs.{amount}")
        print("Amount deposited successfully.")
    else:
        print("Invalid amount.")

# Withdraw money
def withdraw(self, amount):
    if amount <= 0:
        print("Invalid amount.")
    elif amount > self.balance:
        print("Insufficient balance.")
    else:
        self.balance -= amount
        self.transactions.append(f"Withdrawn: Rs.{amount}")
        print("Amount withdrawn successfully.")

# Display balance
def check_balance(self):
    print(f"Current Balance: Rs.{self.balance}")

# Display transaction history
def transaction_history(self):
    print("\nTransaction History:")
```

```
        if len(self.transactions) == 0:
            print("No transactions found.")
        else:
            for transaction in self.transactions:
                print(transaction)

    # Display account details
    def account_details(self):
        print("\nAccount Details")
        print("----------------")
        print("Account Number:", self.account_no)
        print("Account Holder:", self.name)
        print("Balance: Rs.", self.balance)


# Creating object
account = BankAccount(101, "Prince", 5000)

while True:
    print("\n===== BANKING MANAGEMENT SYSTEM =====")
    print("1. Account Details")
    print("2. Deposit")
    print("3. Withdraw")
    print("4. Balance Inquiry")
    print("5. Transaction History")
```

```
print("6. Exit")

choice = input("Enter your choice: ")

if choice == "1":
    account.account_details()

elif choice == "2":
    amount = float(input("Enter deposit amount: Rs."))
    account.deposit(amount)

elif choice == "3":
    amount = float(input("Enter withdrawal amount: Rs."))
    account.withdraw(amount)

elif choice == "4":
    account.check_balance()

elif choice == "5":
    account.transaction_history()

elif choice == "6":
    print("Thank you for using Banking Management System.")
    break
else:
    print("Invalid choice. Please try again.")
```

output:

```
                    python -u "d:\ML_method\pp\ch4.py"
Invalid choice. Please try again.

===== BANKING MANAGEMENT SYSTEM =====
1. Account Details
2. Deposit
3. Withdraw
4. Balance Inquiry
5. Transaction History
6. Exit
Enter your choice: 1

Account Details
----------------
Account Number: 101
Account Holder: Prince
Balance: Rs. 5000

===== BANKING MANAGEMENT SYSTEM =====
1. Account Details
2. Deposit
3. Withdraw
4. Balance Inquiry
5. Transaction History
6. Exit
Enter your choice: 2
Enter deposit amount: Rs.1000000
Amount deposited successfully.

===== BANKING MANAGEMENT SYSTEM =====
1. Account Details
2. Deposit
3. Withdraw
4. Balance Inquiry
5. Transaction History
6. Exit
Enter your choice: 4
Current Balance: Rs.1005000.0

===== BANKING MANAGEMENT SYSTEM =====
1. Account Details
2. Deposit
3. Withdraw
4. Balance Inquiry
5. Transaction History
6. Exit
Enter your choice: 5

Transaction History:
Deposited: Rs.1000000.0

===== BANKING MANAGEMENT SYSTEM =====
1. Account Details
2. Deposit
3. Withdraw
4. Balance Inquiry
5. Transaction History
6. Exit
Enter your choice: 6
Thank you for using Banking Management System.
PS D:\ML_method\pp>
```

Practical 2: Employee Management System

Description:

Create Employee and Manager classes demonstrating inheritance. Calculate salary, bonus,

and department details.

Concepts Covered

● Inheritance

● Polymorphism

Input:

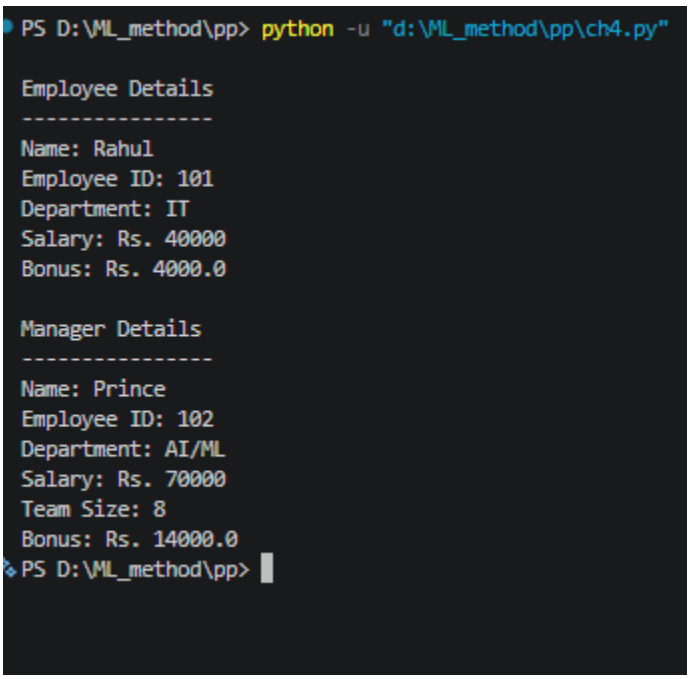
```
class Employee:
  # Constructor
  def __init__(self, name, emp_id, salary, department):
    self.name = name
    self.emp_id = emp_id
    self.salary = salary
    self.department = department
  # Calculate bonus
  def calculate_bonus(self):
    return self.salary * 0.10
  # Display details
  def display_details(self):
    print("\nEmployee Details")
    print("----------------")
    print("Name:", self.name)
    print("Employee ID:", self.emp_id)
```

```
        print("Department:", self.department)
        print("Salary: Rs.", self.salary)
        print("Bonus: Rs.", self.calculate_bonus())
# Manager inherits Employee
class Manager(Employee):

    # Constructor
    def __init__(self, name, emp_id, salary, department, team_size):
        super().__init__(name, emp_id, salary, department)
        self.team_size = team_size
    # Method overriding
    def calculate_bonus(self):
      return self.salary * 0.20
    # Display manager details
    def display_details(self):
        print("\nManager Details")
        print("----------------")
        print("Name:", self.name)
        print("Employee ID:", self.emp_id)
        print("Department:", self.department)
        print("Salary: Rs.", self.salary)
        print("Team Size:", self.team_size)
        print("Bonus: Rs.", self.calculate_bonus())
# Creating objects
employee = Employee(
    "Rahul",
```

```
    101,
    40000,
    "IT"
)
manager = Manager(
    "Prince",
    102,
    70000,
    "AI/ML",
    8
)
# Display details
employee.display_details()
manager.display_details()
```

output:



Practical 3: Hospital Management System

Description:

Develop classes for Patient, Doctor, and Appointment to manage hospital records.

Concepts Covered

- OOP
- Encapsulation

Input:

```
class Patient:
    def __init__(self, patient_id, name, age, disease):
        self.__patient_id = patient_id
        self.__name = name
        self.__age = age
        self.__disease = disease

    # Getter methods
    def get_patient_id(self):
        return self.__patient_id

    def get_name(self):
        return self.__name

    def get_age(self):
        return self.__age

    def get_disease(self):
```

```
        return self.__disease

    def display_details(self):
        print("\nPatient Details")
        print("----------------")
        print("Patient ID:", self.__patient_id)
        print("Name:", self.__name)
        print("Age:", self.__age)
        print("Disease:", self.__disease)
class Doctor:
    def __init__(self, doctor_id, name, specialization):
        self.__doctor_id = doctor_id
        self.__name = name
        self.__specialization = specialization
    def get_name(self):
        return self.__name
    def get_specialization(self):
        return self.__specialization
    def display_details(self):
        print("\nDoctor Details")
        print("----------------")
        print("Doctor ID:", self.__doctor_id)
        print("Name:", self.__name)
        print("Specialization:", self.__specialization)
class Appointment:
    def __init__(self, appointment_id, patient, doctor, date):
```

```
        self.__appointment_id = appointment_id
        self.__patient = patient
        self.__doctor = doctor
        self.__date = date
    def display_appointment(self):
        print("\nAppointment Details")
        print("--------------------")
        print("Appointment ID:", self.__appointment_id)
        print("Patient:", self.__patient.get_name())
        print("Doctor:", self.__doctor.get_name())
        print("Specialization:", self.__doctor.get_specialization())
        print("Date:", self.__date)
# Creating Patient object
patient = Patient(
    101,
    "Rahul",
    25,
    "Fever"
)

# Creating Doctor object
doctor = Doctor(
    201,
    "Dr. Amit",
    "General Physician"
)
```

```
# Creating Appointment object
appointment = Appointment(
    301,
    patient,
    doctor,
    "05-10-2026"
)
# Display records
patient.display_details()
doctor.display_details()
appointment.display_appointment()
```



Practical 4: Online Shopping System

Description:

Develop classes for Customer, Product, Cart, and Order to simulate an online shopping platform.

Concepts Covered

- Objects
- Composition
- Classes

Input:

```
# Practical 4: Online Shopping System
# Concepts: Classes, Objects, Composition



class Product:
    def __init__(self, product_id, name, price):
        self.product_id = product_id
        self.name = name
        self.price = price

    def display_product(self):
        print(f"{self.product_id}. {self.name} - ₹{self.price}")



class Cart:
```

```
    def __init__(self):
        self.products = []   # Composition: Cart contains Product objects

    def add_product(self, product):
        self.products.append(product)
        print(f"{product.name} added to cart.")

    def display_cart(self):
        print("\n--- Shopping Cart ---")

        if not self.products:
            print("Cart is empty.")
            return

        total = 0
        for product in self.products:
            print(f"{product.name} - ₹{product.price}")
            total += product.price

        print(f"Total Amount: ₹{total}")

    def get_total(self):
        return sum(product.price for product in self.products)


class Customer:
```

```
    def __init__(self, customer_id, name):
        self.customer_id = customer_id
        self.name = name
        self.cart = Cart()   # Composition: Customer has a Cart

    def display_customer(self):
        print("\n--- Customer Details ---")
        print("Customer ID:", self.customer_id)
        print("Customer Name:", self.name)


class Order:
    def __init__(self, order_id, customer):
        self.order_id = order_id
        self.customer = customer
        self.amount = customer.cart.get_total()

    def place_order(self):
        print("\n--- Order Details ---")
        print("Order ID:", self.order_id)
        print("Customer:", self.customer.name)
        print("Order Amount: ₹", self.amount)
        print("Order placed successfully!")


# Creating Product objects
```

```
p1 = Product(1, "Laptop", 55000)
p2 = Product(2, "Mouse", 800)
p3 = Product(3, "Keyboard", 1500)

# Display products
print("--- Available Products ---")
p1.display_product()
p2.display_product()
p3.display_product()

# Creating Customer object
customer = Customer(101, "Prince")

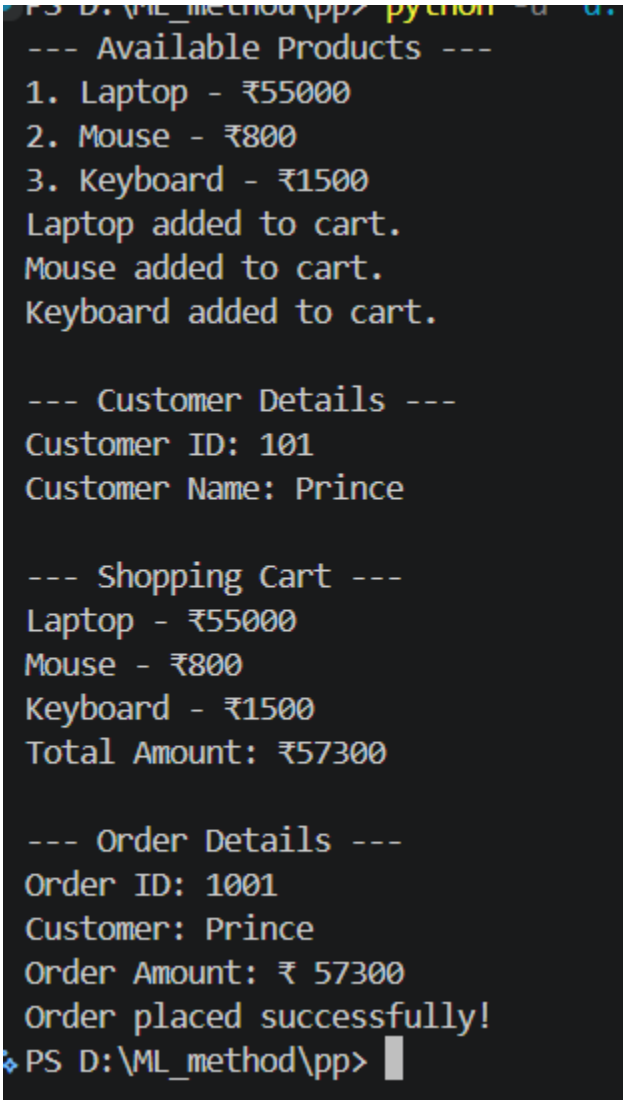
# Add products to customer's cart
customer.cart.add_product(p1)
customer.cart.add_product(p2)
customer.cart.add_product(p3)

# Display customer details
customer.display_customer()

# Display cart
customer.cart.display_cart()

# Create and place order
order = Order(1001, customer)
```

order.place_order()



Practical 5: Vehicle Rental System

Description:

Implement classes for different vehicle types demonstrating inheritance and method overriding.

Concepts Covered

● Inheritance

● Method Overriding

Input:

```
# Practical 5: Vehicle Rental System
# Concepts: Inheritance and Method Overriding



# Parent class
class Vehicle:
    def __init__(self, vehicle_no, brand, rent):
        self.vehicle_no = vehicle_no
        self.brand = brand
        self.rent = rent

    def display_details(self):
        print("Vehicle No:", self.vehicle_no)
        print("Brand:", self.brand)
        print("Rent per day: ₹", self.rent)

    def calculate_rent(self, days):
        return self.rent * days



# Child class 1
class Car(Vehicle):
    def calculate_rent(self, days):
```

```
        # Method overriding
        return (self.rent * days) + 500


# Child class 2
class Bike(Vehicle):
    def calculate_rent(self, days):
        # Method overriding
        return self.rent * days


# Child class 3
class Truck(Vehicle):
    def calculate_rent(self, days):
        # Method overriding
        return (self.rent * days) + 1000


# Creating objects
car = Car("GJ01AB1234", "Toyota", 2000)
bike = Bike("GJ01CD5678", "Honda", 800)
truck = Truck("GJ01EF9012", "Tata", 3000)

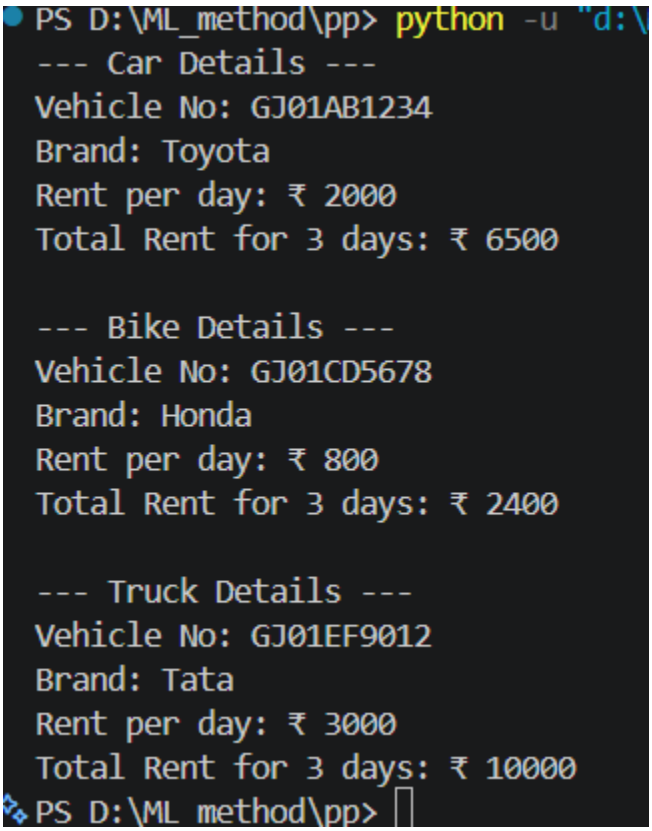
days = 3

# Car details
```

```
print("--- Car Details ---")
car.display_details()
print("Total Rent for", days, "days: ₹", car.calculate_rent(days))

# Bike details
print("\n--- Bike Details ---")
bike.display_details()
print("Total Rent for", days, "days: ₹", bike.calculate_rent(days))

# Truck details
print("\n--- Truck Details ---")
truck.display_details()
print("Total Rent for", days, "days: ₹", truck.calculate_rent(days))
```



Practical 6: College Management System

Description:

Develop a complete college management system using multiple classes such as Student, Faculty, Course, and Department.

Concepts Covered

● OOP Design

● Information Hiding

```
# Practical 6: College Management System
# Concepts: OOP Design and Information Hiding



class Student:
    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name
        self.__marks = 0      # Private variable

    def set_marks(self, marks):
        if 0 <= marks <= 100:
            self.__marks = marks
        else:
            print("Invalid marks!")

    def get_marks(self):
        return self.__marks
```

```
    def display(self):
        print("Student ID:", self.student_id)
        print("Student Name:", self.name)
        print("Marks:", self.__marks)


class Faculty:
    def __init__(self, faculty_id, name, subject):
        self.faculty_id = faculty_id
        self.name = name
        self.subject = subject

    def display(self):
        print("Faculty ID:", self.faculty_id)
        print("Faculty Name:", self.name)
        print("Subject:", self.subject)


class Course:
    def __init__(self, course_id, course_name):
        self.course_id = course_id
        self.course_name = course_name

    def display(self):
        print("Course ID:", self.course_id)
```

```
        print("Course Name:", self.course_name)



class Department:
    def __init__(self, department_id, name):
        self.department_id = department_id
        self.name = name
        self.students = []
        self.faculties = []
        self.courses = []
    def add_student(self, student):
        self.students.append(student)
    def add_faculty(self, faculty):
        self.faculties.append(faculty)
    def add_course(self, course):
        self.courses.append(course)
    def display_department(self):
        print("\n--- Department Details ---")
        print("Department ID:", self.department_id)
        print("Department Name:", self.name)
        print("\nStudents:")
        for student in self.students:
            student.display()

        print("\nFaculty:")
        for faculty in self.faculties:
```

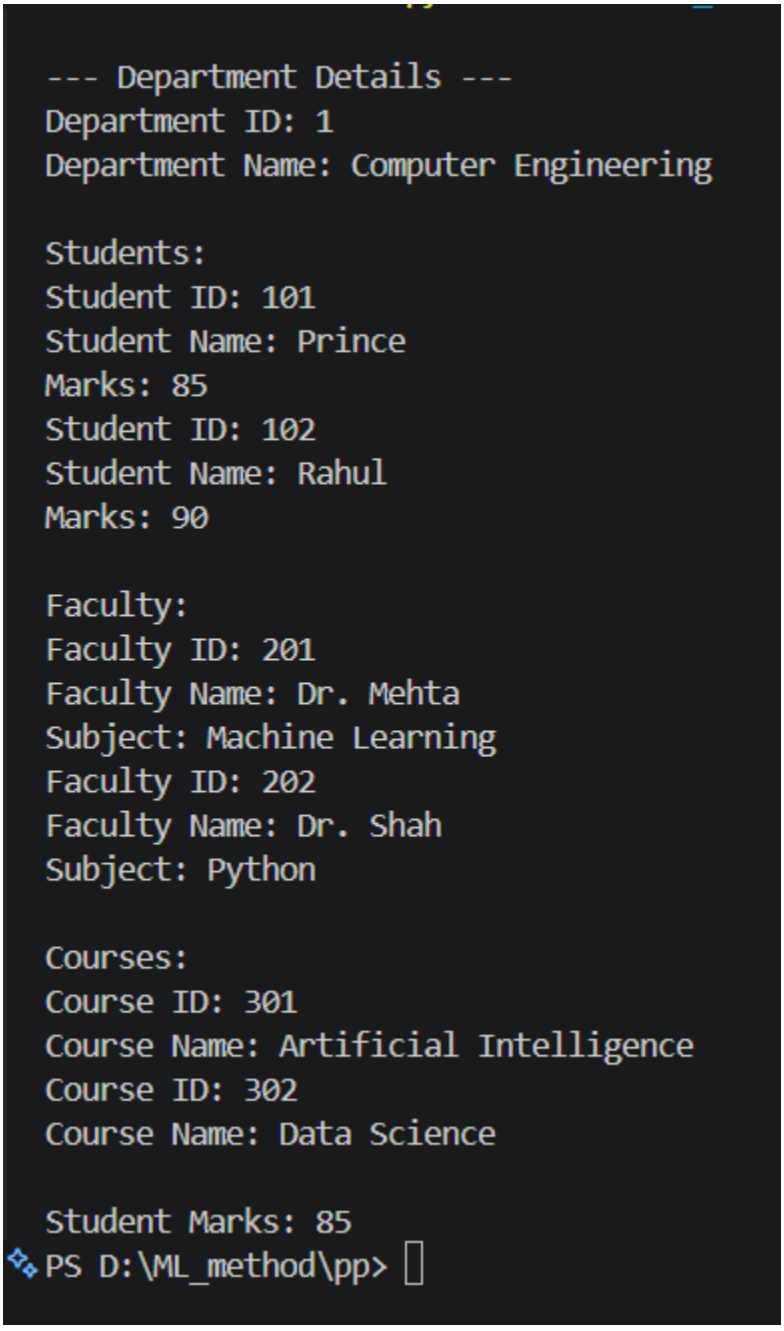
```
        faculty.display()

    print("\nCourses:")
    for course in self.courses:
        course.display()


# Creating objects
student1 = Student(101, "Prince")
student1.set_marks(85)
student2 = Student(102, "Rahul")
student2.set_marks(90)
faculty1 = Faculty(201, "Dr. Mehta", "Machine Learning")
faculty2 = Faculty(202, "Dr. Shah", "Pyton")
course1 = Course(301, "Artificial Intelligence")
course2 = Course(302, "Data Science")
department = Department(1, "Computer Engineering")
# Adding students, faculty and courses
department.add_student(student1)
department.add_student(student2)
department.add_faculty(faculty1)
department.add_faculty(faculty2)
department.add_course(course1)
department.add_course(course2)
# Display complete department information
department.display_department()
```

Information Hiding demonstration

print("\nStudent Marks:", student1.get_marks())



Unit 5: Data Science and Data

Visualization

Practical 1: Student Performance Analysis using Pandas

Description:

Import student marks from a CSV file into a DataFrame. Calculate average marks, highest marks, and generate department-wise statistics.

Concepts Covered

- Pandas
- DataFrames

Input:

```python
import pandas as pd

# Import CSV file into DataFrame
df = pd.read_csv("student_marks.csv")

# Display student data
print("--- Student Data ---")
print(df)

# Calculate average marks
average_marks = df["Marks"].mean()
print("\nAverage Marks:", average_marks)

# Find highest marks
highest_marks = df["Marks"].max()
print("Highest Marks:", highest_marks)

# Find student with highest marks
top_student = df.loc[df["Marks"].idxmax()]
print("Top Student:", top_student["Name"])

# Department-wise statistics
print("\n--- Department-wise Statistics ---")

department_stats = df.groupby("Department")["Marks"].agg(
    ["mean", "max", "min"]
)

print(department_stats)
```

✓ 35.3s

```
--- Student Data ---
     Name Department  Marks
0  Prince      AI-ML     85
1   Rahul   Computer     78
2    Riya      AI-ML     92
3    Amit   Computer     88
4    Neha         IT     75
5     Jay      AI-ML     80

Average Marks: 83.0
Highest Marks: 92
Top Student: Riya

--- Department-wise Statistics ---
                mean  max  min
Department
AI-ML       85.666667   92   80
Computer    83.000000   88   78
IT          75.000000   75   75
```

Practical 2: COVID-19 Data Visualization

Description:

Analyze COVID-19 data stored in a CSV file and generate bar charts, pie charts, and line graphs for confirmed, recovered, and active cases.

Concepts Covered

- Pandas
- Matplotlib

```python
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("covid_data.csv")

print("--- COVID-19 Data ---")
print(df)

plt.figure(figsize=(8, 5))

plt.bar(df["Country"], df["Confirmed"])

plt.xlabel("Country")
plt.ylabel("Confirmed Cases")
plt.title("Confirmed COVID-19 Cases")

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

total_confirmed = df["Confirmed"].sum()
total_recovered = df["Recovered"].sum()
total_active = df["Active"].sum()

values = [total_confirmed, total_recovered, total_active]
labels = ["Confirmed", "Recovered", "Active"]

plt.figure(figsize=(7, 7))

plt.pie(values, labels=labels, autopct="%1.1f%%")

plt.title("COVID-19 Case Distribution")
plt.show()

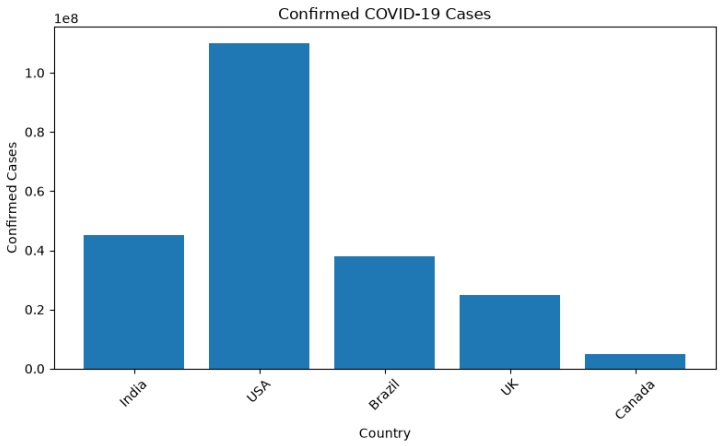
plt.figure(figsize=(8, 5))

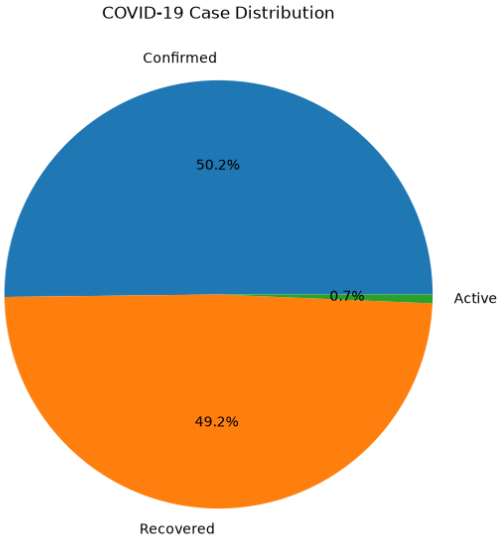
plt.plot(df["Country"], df["Confirmed"], marker="o", label="Confirmed")
plt.plot(df["Country"], df["Recovered"], marker="o", label="Recovered")
plt.plot(df["Country"], df["Active"], marker="o", label="Active")

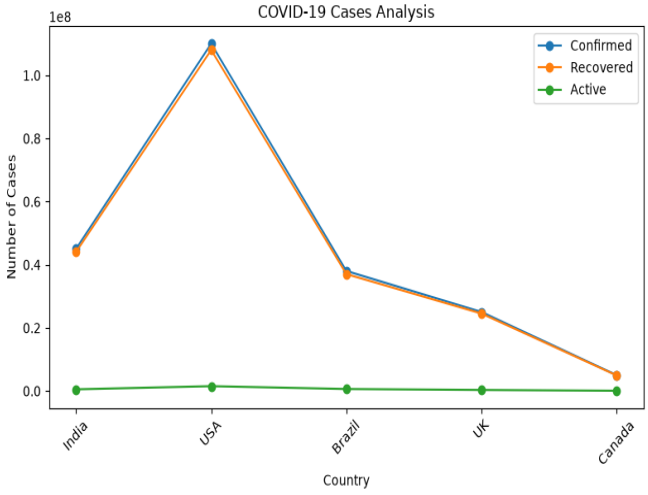
plt.xlabel("Country")
plt.ylabel("Number of Cases")
plt.title("COVID-19 Cases Analysis")

plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
--- COVID-19 Data ---
   Country  Confirmed  Recovered   Active
0    India   45000000   44000000   500000
1      USA  110000000  108000000  1500000
2   Brazil   38000000   37000000   600000
3       UK   25000000   24500000   300000
4   Canada    5000000    4900000    50000
```







Practical 3: Sales Dashboard

Description:

Create a dashboard that visualizes monthly product sales using multiple graph types.

Concepts Covered

- DataFrames
- Graph Plotting

```python
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv("sales_data.csv")

print("--- Sales Data ---")
print(df)

df["Total_Sales"] = df["Quantity"] * df["Price"]

print("\n--- Sales Data with Total Sales ---")
print(df)

product_sales = df.groupby("Product")["Total_Sales"].sum()

plt.figure(figsize=(8, 5))
plt.bar(product_sales.index, product_sales.values)

plt.xlabel("Product")
plt.ylabel("Sales Amount")
plt.title("Product-wise Sales")

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

monthly_sales = df.groupby("Month")["Total_Sales"].sum()

plt.figure(figsize=(8, 5))
plt.plot(monthly_sales.index, monthly_sales.values, marker="o")

plt.xlabel("Month")
plt.ylabel("Sales Amount")
plt.title("Monthly Sales")

plt.tight_layout()
plt.show()

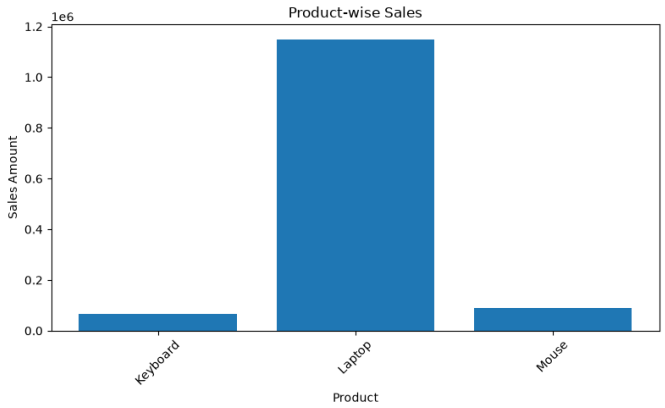
plt.figure(figsize=(7, 7))
plt.pie(
    product_sales.values,
    labels=product_sales.index,
    autopct="%1.1f%%"
)

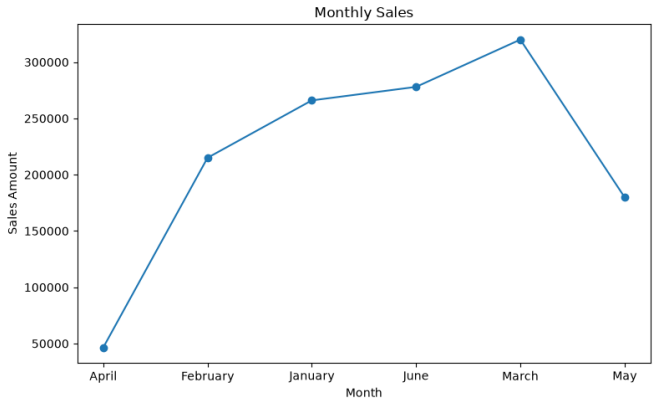
plt.title("Product-wise Sales Distribution")
plt.show()

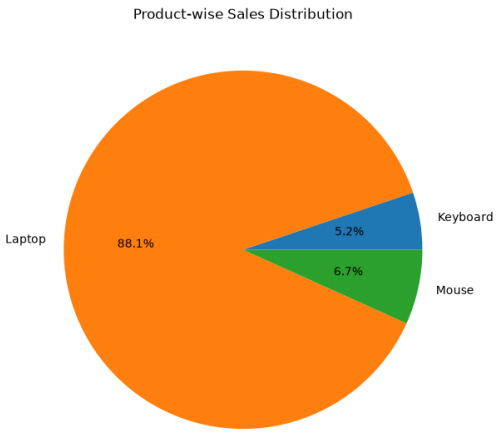
print("\nTotal Sales: ₹", df["Total_Sales"].sum())
```

```
--- Sales Data ---
       Month   Product  Quantity  Price
0    January    Laptop         5  50000
1    January     Mouse        20    800
2   February    Laptop         4  50000
3   February  Keyboard        10   1500
4      March     Mouse        25    800
5      March    Laptop         6  50000
6      April  Keyboard        15   1500
7      April     Mouse        30    800
8        May    Laptop         3  50000
9        May  Keyboard        20   1500
10      June     Mouse        35    800
11      June    Laptop         5  50000

--- Sales Data with Total Sales ---
       Month   Product  Quantity  Price  Total_Sales
0    January    Laptop         5  50000       250000
1    January     Mouse        20    800        16000
2   February    Laptop         4  50000       200000
3   February  Keyboard        10   1500        15000
4      March     Mouse        25    800        20000
5      March    Laptop         6  50000       300000
6      April  Keyboard        15   1500        22500
7      April     Mouse        30    800        24000
8        May    Laptop         3  50000       150000
9        May  Keyboard        20   1500        30000
10      June     Mouse        35    800        28000
11      June    Laptop         5  50000       250000
```







Practical 4: IPL Statistics Analyzer

Description:

Analyze IPL player performance and visualize runs, wickets, strike rates, and team-wise statistics.

Concepts Covered

- Data Analysis
- Charts

```python
import pandas as pd
import matplotlib.pyplot as plt

# Read IPL data from CSV file
df = pd.read_csv("ipl_players.csv")

print("--- IPL Player Data ---")
print(df)
top_runs = df.loc[df["Runs"].idxmax()]
print("\n--- Highest Run Scorer ---")
print("Player:", top_runs["Player"])
print("Runs:", top_runs["Runs"])
top_wickets = df.loc[df["Wickets"].idxmax()]
print("\n--- Highest Wicket Taker ---")
print("Player:", top_wickets["Player"])
print("Wickets:", top_wickets["Wickets"])
average_sr = df["Strike_Rate"].mean()

print("\nAverage Strike Rate:", round(average_sr, 2))

plt.figure(figsize=(9, 5))

plt.bar(df["Player"], df["Runs"])

plt.xlabel("Player")
plt.ylabel("Runs")
plt.title("IPL Player Runs")

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
plt.figure(figsize=(9, 5))

plt.bar(df["Player"], df["Wickets"])

plt.xlabel("Player")
plt.ylabel("Wickets")
plt.title("IPL Player Wickets")

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
plt.figure(figsize=(9, 5))

plt.plot(df["Player"], df["Strike_Rate"], marker="o")

plt.xlabel("Player")
plt.ylabel("Strike Rate")
plt.title("IPL Player Strike Rate")

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
team_stats = df.groupby("Team").agg({
    "Runs": "sum",
    "Wickets": "sum",
    "Strike_Rate": "mean"
})

print("\n--- Team-wise Statistics ---")
print(team_stats)
plt.figure(figsize=(8, 5))
plt.bar(team_stats.index, team_stats["Runs"])
plt.xlabel("Team")
plt.ylabel("Total Runs")
plt.title("Team-wise Total Runs")

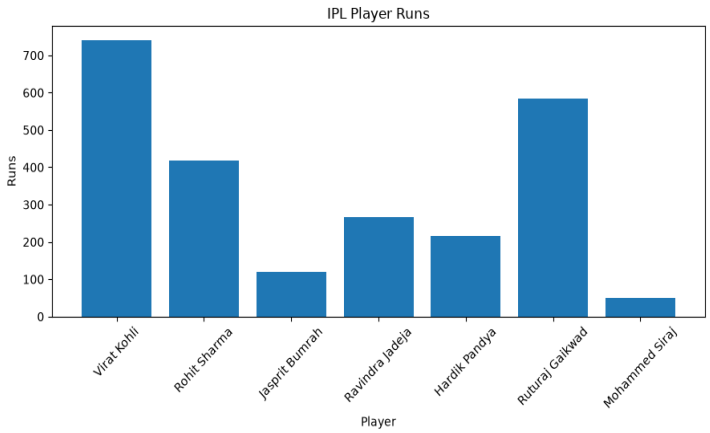
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

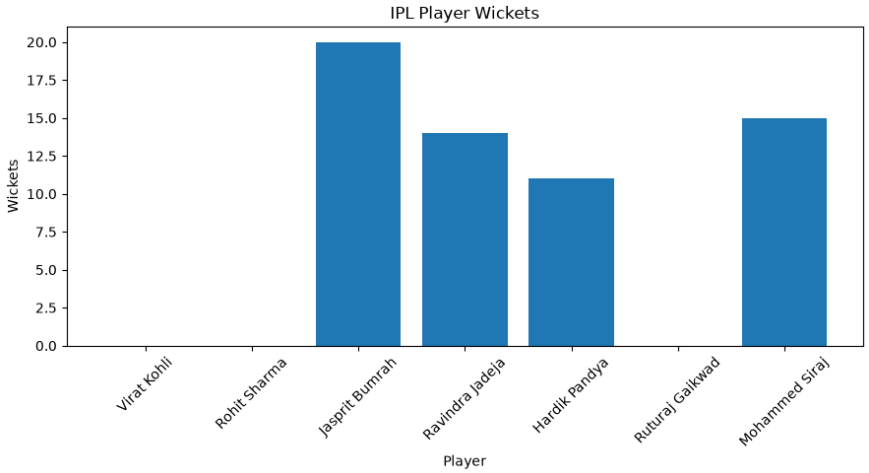
```
--- IPL Player Data ---
            Player Team  Runs  Wickets  Strike_Rate
0      Virat Kohli  RCB   741        0        154.7
1     Rohit Sharma   MI   417        0        131.2
2   Jasprit Bumrah   MI   120       20        105.3
3  Ravindra Jadeja  CSK   267       14        142.1
4    Hardik Pandya   MI   216       11        143.5
5  Ruturaj Gaikwad  CSK   583        0        147.5
6   Mohammed Siraj  RCB    50       15        110.2

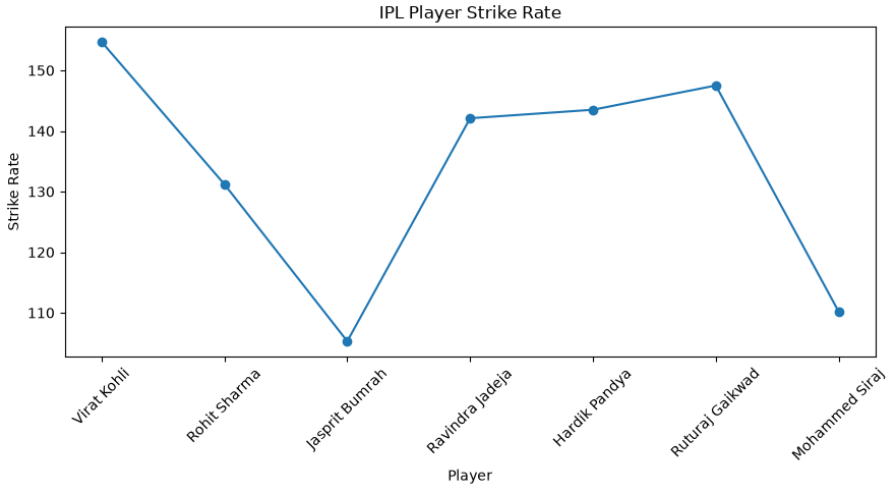
--- Highest Run Scorer ---
Player: Virat Kohli
Runs: 741

--- Highest Wicket Taker ---
Player: Jasprit Bumrah
Wickets: 20

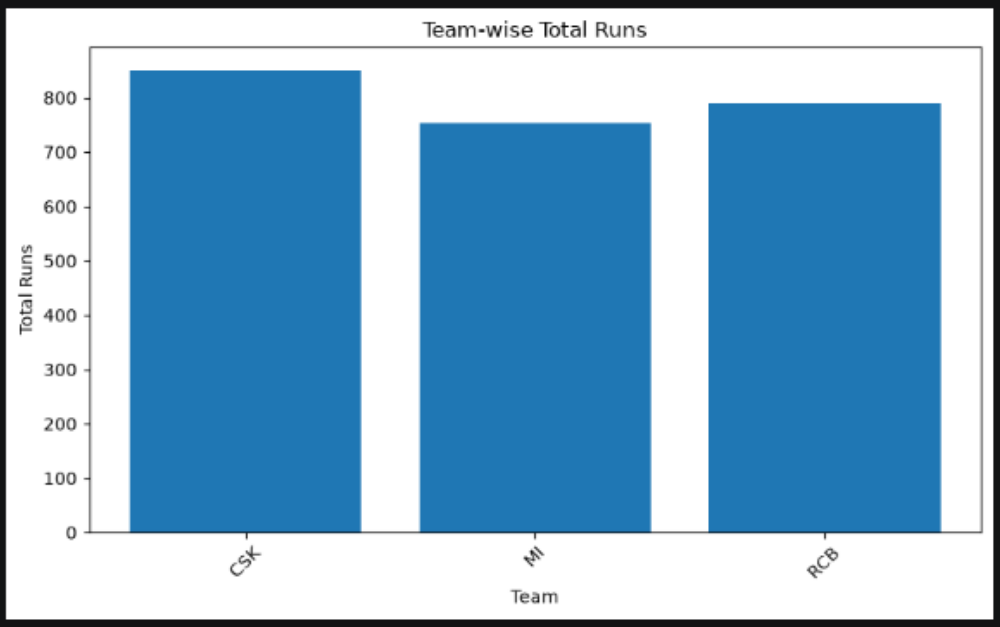
Average Strike Rate: 133.5
```







```
--- Team-wise Statistics ---
      Runs  Wickets  Strike_Rate
Team
CSK    850       14   144.800000
MI     753       31   126.666667
RCB    791       15   132.450000
```



Practical 5: Weather Data Visualization

Description:

Visualize daily temperature, rainfall, and humidity data using line charts, histograms, and bar

charts.

Concepts Covered

● Plotting

● Histograms

```python
import pandas as pd
import matplotlib.pyplot as plt

# Read weather data from CSV file
df = pd.read_csv("weather.csv")

# Display data
print("--- Weather Data ---")
print(df)

# 1. Line Chart - Daily Temperature
plt.figure(figsize=(9, 5))
plt.plot(df["Day"], df["Temperature"], marker="o")
plt.xlabel("Day")
plt.ylabel("Temperature (°C)")
plt.title("Daily Temperature")
plt.grid(True)
plt.show()

# 2. Line Chart - Daily Humidity
plt.figure(figsize=(9, 5))
plt.plot(df["Day"], df["Humidity"], marker="o")
plt.xlabel("Day")
plt.ylabel("Humidity (%)")
plt.title("Daily Humidity")
plt.grid(True)
plt.show()

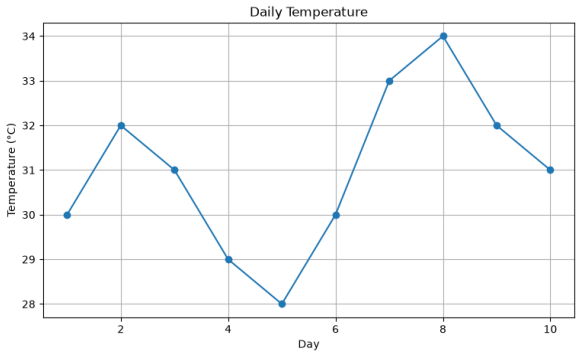
# 3. Bar Chart - Daily Rainfall
plt.figure(figsize=(9, 5))
plt.bar(df["Day"], df["Rainfall"])
plt.xlabel("Day")
plt.ylabel("Rainfall (mm)")
plt.title("Daily Rainfall")
plt.show()

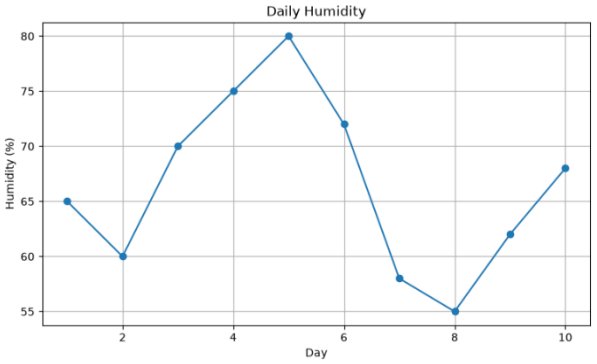
# 4. Histogram - Temperature Distribution
plt.figure(figsize=(8, 5))
plt.hist(df["Temperature"], bins=5, edgecolor="black")
plt.xlabel("Temperature (°C)")
plt.ylabel("Frequency")
plt.title("Temperature Distribution")
plt.show()

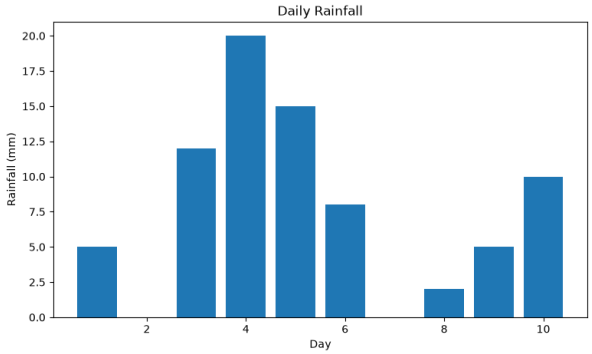

plt.figure(figsize=(8, 5))
plt.hist(df["Humidity"], bins=5, edgecolor="black")
plt.xlabel("Humidity (%)")
plt.ylabel("Frequency")
plt.title("Humidity Distribution")
plt.show()

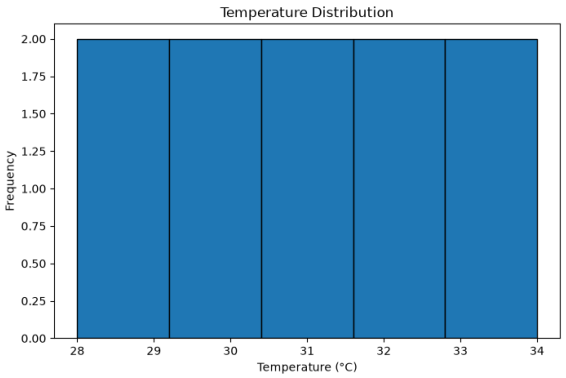
print("\n--- Weather Statistics ---")
print("Average Temperature:", round(df["Temperature"].mean(), 2), "°C")
print("Maximum Temperature:", df["Temperature"].max(), "°C")
print("Average Rainfall:", round(df["Rainfall"].mean(), 2), "mm")
print("Average Humidity:", round(df["Humidity"].mean(), 2), "%")
```

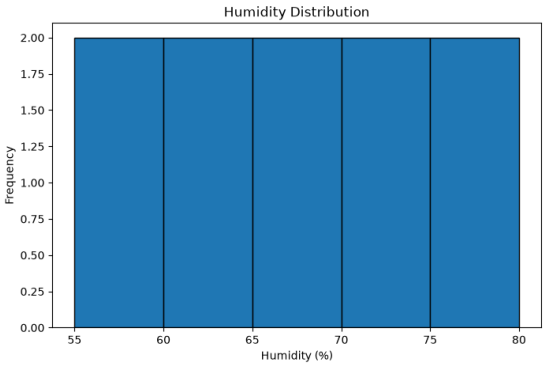
```
--- Weather Data ---
   Day  Temperature  Rainfall  Humidity
0    1           30         5        65
1    2           32         0        60
2    3           31        12        70
3    4           29        20        75
4    5           28        15        80
5    6           30         8        72
6    7           33         0        58
7    8           34         2        55
8    9           32         5        62
```












```
--- Weather Statistics ---
Average Temperature: 31.0 °C
Maximum Temperature: 34 °C
Average Rainfall: 7.7 mm
Average Humidity: 66.5 %
```

Practical 6: Stock Market Trend Analysis

Description:

Read stock prices from CSV files and visualize opening price, closing price, trading volume, and moving averages.

Concepts Covered

- Data Visualization
- Line Graphs

```python
import pandas as pd
import matplotlib.pyplot as plt

# Read stock data from CSV file
df = pd.read_csv("stock_data.csv")

# Convert Date column into date format
df["Date"] = pd.to_datetime(df["Date"])

# Display data
print("--- Stock Market Data ---")
print(df)

# Calculate 3-day moving average of closing price
df["Moving_Average"] = df["Close"].rolling(window=3).mean()

# 1. Opening Price and Closing Price
plt.figure(figsize=(10, 5))
plt.plot(df["Date"], df["Open"], marker="o", label="Opening Price")
plt.plot(df["Date"], df["Close"], marker="o", label="Closing Price")

plt.xlabel("Date")
plt.ylabel("Price")
plt.title("Opening and Closing Stock Prices")
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# 2. Trading Volume
plt.figure(figsize=(10, 5))
plt.plot(df["Date"], df["Volume"], marker="o")

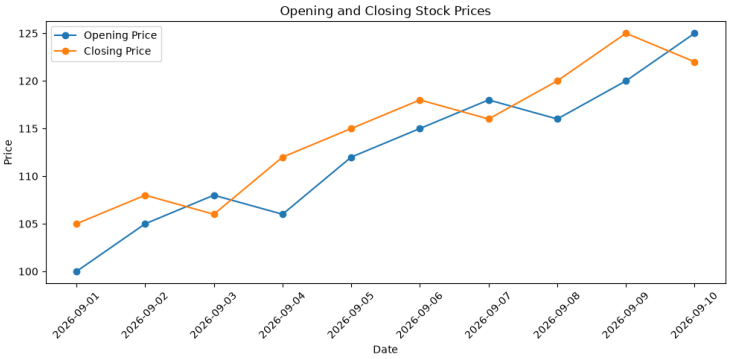
plt.xlabel("Date")
plt.ylabel("Trading Volume")
plt.title("Trading Volume")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

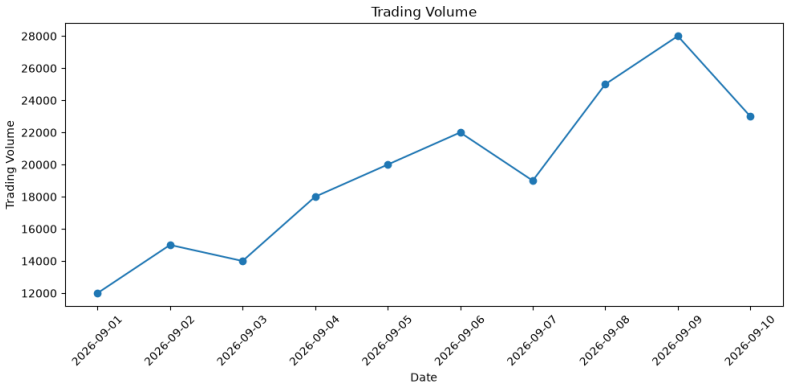
# 3. Closing Price and Moving Average
plt.figure(figsize=(10, 5))
plt.plot(df["Date"], df["Close"], marker="o", label="Closing Price")
plt.plot(df["Date"], df["Moving_Average"], marker="o",
         label="3-Day Moving Average")

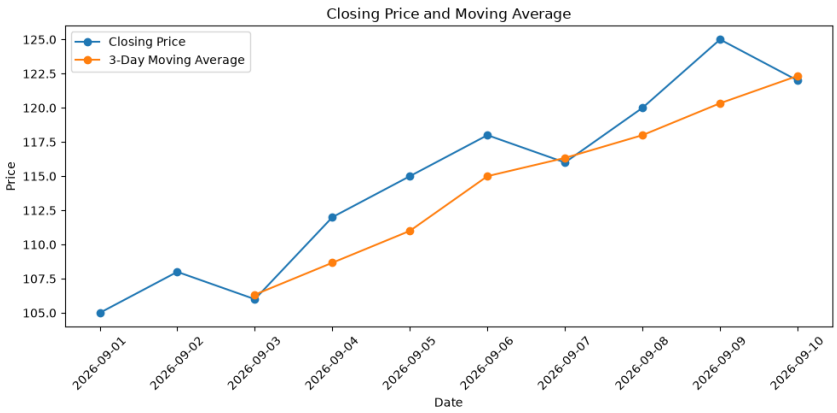
plt.xlabel("Date")
plt.ylabel("Price")
plt.title("Closing Price and Moving Average")
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Display final data
print("\n--- Stock Data with Moving Average ---")
print(df)
```

```
--- Stock Market Data ---
        Date  Open  Close  Volume
0 2026-09-01   100    105   12000
1 2026-09-02   105    108   15000
2 2026-09-03   108    106   14000
3 2026-09-04   106    112   18000
4 2026-09-05   112    115   20000
5 2026-09-06   115    118   22000
6 2026-09-07   118    116   19000
7 2026-09-08   116    120   25000
8 2026-09-09   120    125   28000
9 2026-09-10   125    122   23000
```








```
--- Stock Data with Moving Average ---
        Date  Open  Close  Volume  Moving_Average
0 2026-09-01   100    105   12000             NaN
1 2026-09-02   105    108   15000             NaN
2 2026-09-03   108    106   14000      106.333333
3 2026-09-04   106    112   18000      108.666667
4 2026-09-05   112    115   20000      111.000000
5 2026-09-06   115    118   22000      115.000000
6 2026-09-07   118    116   19000      116.333333
7 2026-09-08   116    120   25000      118.000000
8 2026-09-09   120    125   28000      120.333333
9 2026-09-10   125    122   23000      122.333333
```

Unit 6: Regular Expressions

Practical 1: Email Address Validator

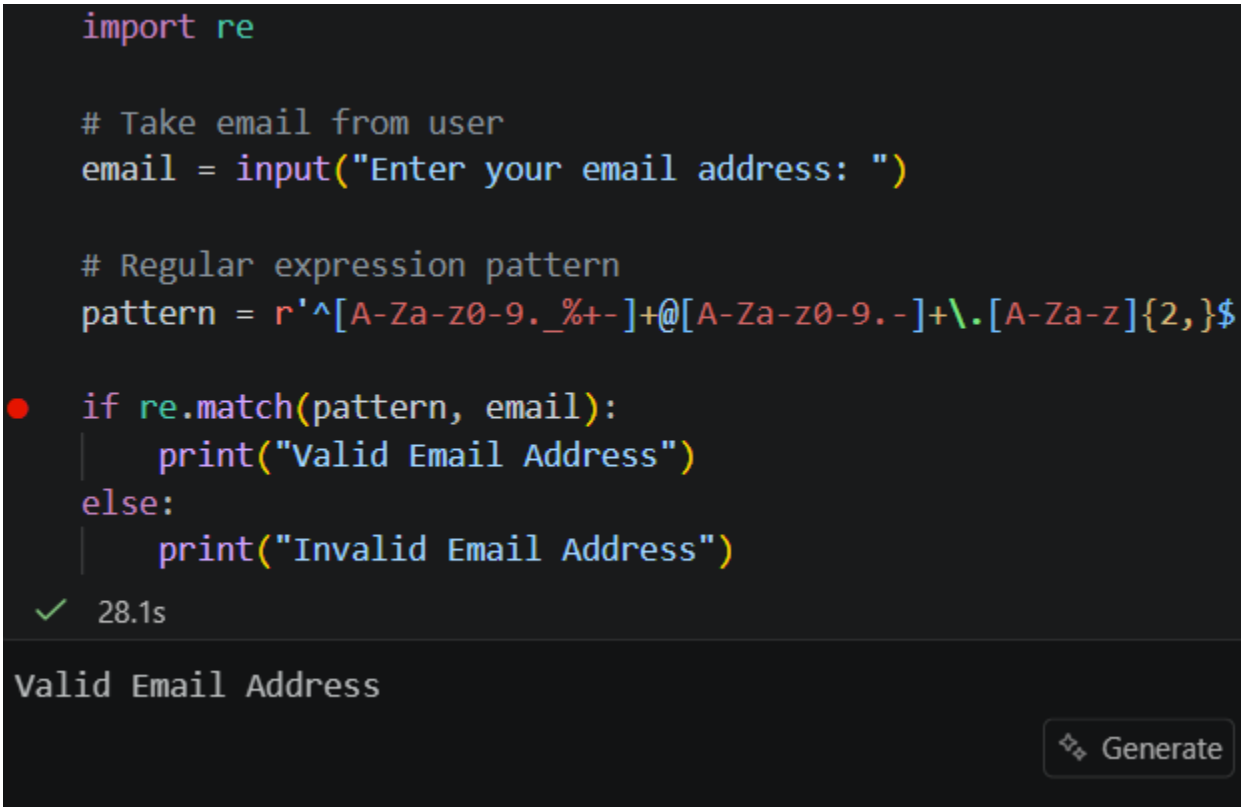
Description:

Validate email addresses entered by users using regular expressions.

Concepts Covered

● Regular Expressions

● Pattern Matching



Email was princemoradiya2020@gmail.com

Practical 2: Mobile Number Verification System

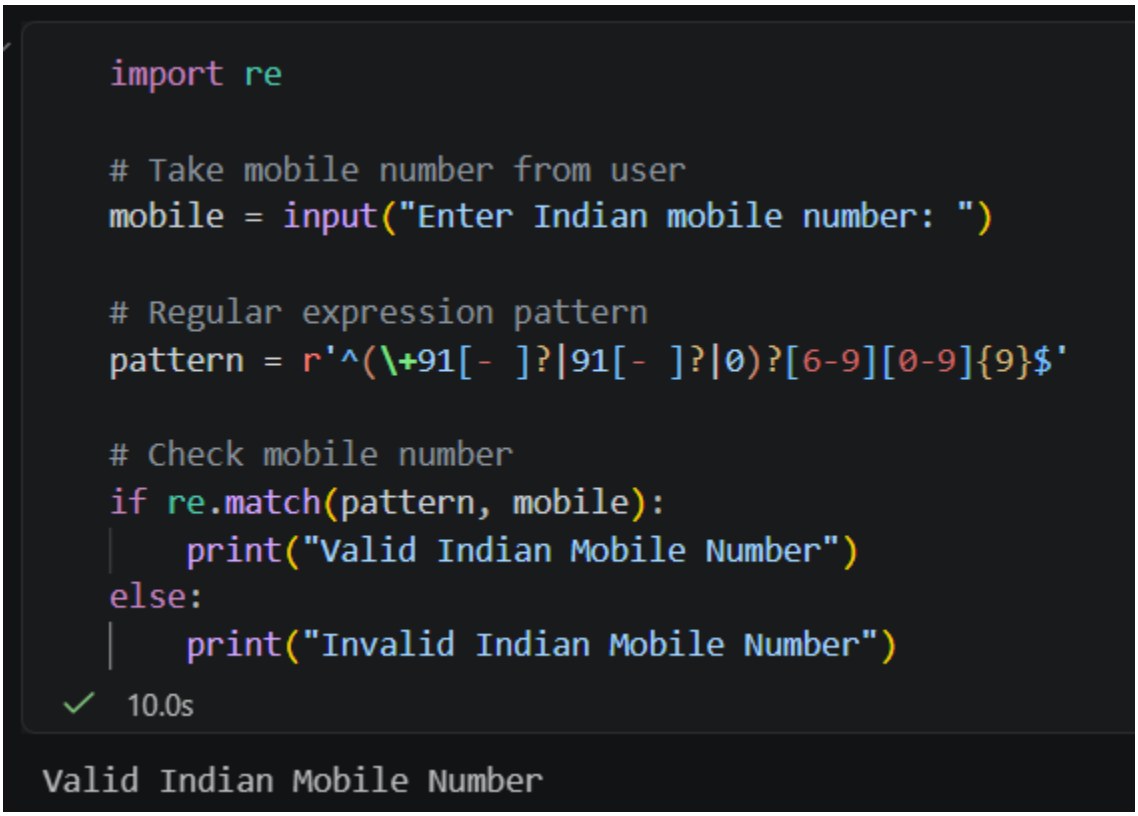
Description:

Validate Indian mobile numbers with country codes using regular expressions.

Concepts Covered

● Regex Quantifiers

● Character Classes



Practical 3: Password Policy Checker

Description:

Verify whether passwords satisfy organizational security policies using regular expressions.

Concepts Covered

● Regex

● Special Characters

```python
import re

# Take password from user
password = input("Enter your password: ")

# Password policy:
# Minimum 8 characters
# At least one uppercase letter
# At least one lowercase letter
# At least one digit
# At least one special character

pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*[0-9])(?=.*[@#$%^&*!])[A-Za-z0-9@#$%^&*!]{8,}$'

# Check password
if re.match(pattern, password):
    print("Valid Password")
else:
    print("Invalid Password")
```

✓ 2.4s

```
Invalid Password
```

Practical 4: HTML Information Extractor

Description:

Extract hyperlinks, email addresses, and headings from HTML pages using regular expressions.

Concepts Covered

- Regex
- HTML Parsing

```python
import re

# Sample HTML content
html = """
<html>
<head>
    <title>My Web Page</title>
</head>
<body>

<h1>Welcome to My Website</h1>
<h2>About Us</h2>
<h2>Contact Information</h2>

<p>Email: prince@gmail.com</p>
<p>Another email: support@example.com</p>

<a href="https://www.google.com">Google</a>
<a href="https://www.python.org">Python</a>
<a href="https://www.github.com">GitHub</a>

</body>
</html>
"""

# 1. Extract hyperlinks
links = re.findall(r'href=["\'](.*?)["\']', html)

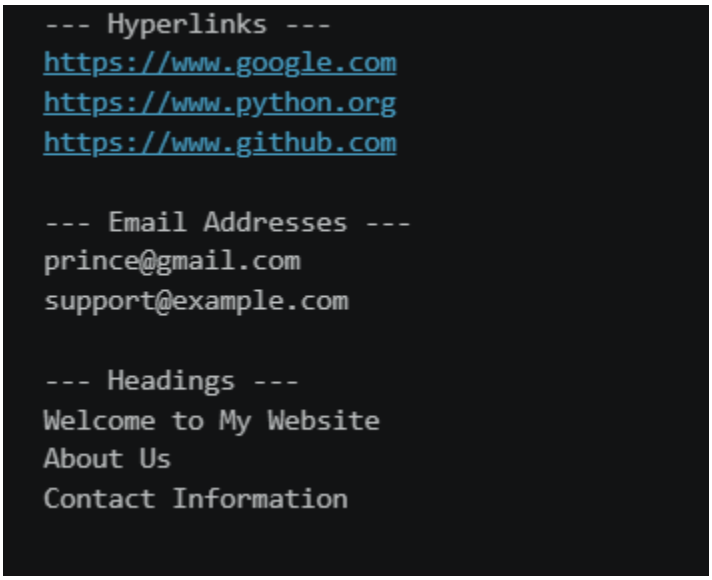
# 2. Extract email addresses
emails = re.findall(
    r'[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}',
    html
)

# 3. Extract headings
headings = re.findall(r'<h[1-6]>(.*?)</h[1-6]>', html)

# Display results
print("--- Hyperlinks ---")
for link in links:
    print(link)

print("\n--- Email Addresses ---")
for email in emails:
    print(email)

print("\n--- Headings ---")
for heading in headings:
    print(heading)
```



Practical 5: Resume Information Extractor

Description:

Extract candidate name, phone number, email address, and skills from resume text files using

regular expressions.

Concepts Covered

- Regex
- File Handling

```python
import re

# Open and read resume file
with open("resume.txt", "r") as file:
    resume = file.read()

# Extract candidate name
name = re.search(r'Name:\s*(.*)', resume)

# Extract phone number
phone = re.search(r'Phone:\s*(\+91[- ]?[6-9][0-9]{9})', resume)

# Extract email address
email = re.search(
    r'Email:\s*([A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,})',
    resume
)

# Extract skills
skills = re.search(r'Skills:\s*\n(.*)', resume)

# Display extracted information
print("--- Resume Information ---")

if name:
    print("Candidate Name:", name.group(1))

if phone:
    print("Phone Number:", phone.group(1))

if email:
    print("Email Address:", email.group(1))

if skills:
    skill_list = skills.group(1).split(",")
    print("Skills:")

    for skill in skill_list:
        print("-", skill.strip())
```

```
--- Resume Information ---
Candidate Name: Prince Moradiya
Phone Number: +91 9876543210
Email Address: prince@gmail.com
Skills:
- Python
- SQL
- Machine Learning
- Data Science
- HTML
- CSS
```

Practical 6: Web Log Pattern Analyzer

Description:

Analyze web server log files and extract IP addresses, timestamps, request methods, and status codes using regular expressions.

Concepts Covered

- Regex
- File Processing

```python
import re

# Open and read log file
with open("web_log.txt", "r") as file:
    logs = file.readlines()

# Regular expression pattern
pattern = (
    r'(\d+\.\d+\.\d+\.\d+).*'
    r'\[([^\]]+)\].*'
    r'"(GET|POST|PUT|DELETE|PATCH|HEAD|OPTIONS)'
    r'\s+[^\s]+\s+HTTP/[0-9.]+"'
    r'\s+(\d{3})'
)

print("--- Web Log Analysis ---")

# Process each log line
for log in logs:

    match = re.search(pattern, log)

    if match:
        ip_address = match.group(1)
        timestamp = match.group(2)
        method = match.group(3)
        status_code = match.group(4)

        print("\nIP Address:", ip_address)
        print("Timestamp:", timestamp)
        print("Request Method:", method)
        print("Status Code:", status_code)
```

```
--- Web Log Analysis ---

IP Address: 192.168.1.10
Timestamp: 01/Oct/2026:10:15:20 +0530
Request Method: GET
Status Code: 200

IP Address: 192.168.1.15
Timestamp: 01/Oct/2026:10:16:05 +0530
Request Method: POST
Status Code: 201

IP Address: 10.0.0.5
Timestamp: 01/Oct/2026:10:17:10 +0530
Request Method: GET
Status Code: 200

IP Address: 172.16.0.8
Timestamp: 01/Oct/2026:10:18:45 +0530
Request Method: DELETE
Status Code: 404

IP Address: 192.168.1.20
Timestamp: 01/Oct/2026:10:19:30 +0530
Request Method: GET
Status Code: 500
```
\

Unit 7: Python Database Connectivity

Practical 1: Student Database Management System

Description:
Develop a Python application that performs Create, Read, Update, and Delete (CRUD)
operations on a MySQL student database.

Concepts Covered

- MySQL Connector
- CRUD Operations

CODE :-

```sql
CREATE DATABASE college;

USE college;

CREATE TABLE student (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT,
    course VARCHAR(50),
    marks FLOAT
);
```

```python
import mysql.connector

# Connect to MySQL
con = mysql.connector.connect(
    host="localhost",
    user="root",
    password="MySQL@12345",
    database="college"
)

cursor = con.cursor()
```

```python
# CREATE - Add Student
def add_student():
    student_id = int(input("Enter Student ID: "))
    name = input("Enter Student Name: ")
    age = int(input("Enter Age: "))
    course = input("Enter Course: ")
    marks = float(input("Enter Marks: "))

    query = """
    INSERT INTO student (id, name, age, course, marks)
    VALUES (%s, %s, %s, %s, %s)
    """

    values = (student_id, name, age, course, marks)

    cursor.execute(query, values)
    con.commit()

    print("Student added successfully!")


# READ - Display Students
def display_students():
    cursor.execute("SELECT * FROM student")

    records = cursor.fetchall()

    if len(records) == 0:
        print("No student records found.")
    else:
        print("\n----- Student Records -----")

        for student in records:
            print(
                "ID:", student[0],
                "| Name:", student[1],
                "| Age:", student[2],
                "| Course:", student[3],
                "| Marks:", student[4]
            )


# UPDATE - Update Student
def update_student():
    student_id = int(input("Enter Student ID to update: "))

    name = input("Enter New Name: ")
    age = int(input("Enter New Age: "))
    course = input("Enter New Course: ")
    marks = float(input("Enter New Marks: "))

    query = """
    UPDATE student
    SET name=%s, age=%s, course=%s, marks=%s
    WHERE id=%s
```

```
    """

    values = (name, age, course, marks, student_id)

    cursor.execute(query, values)
    con.commit()

    if cursor.rowcount > 0:
        print("Student updated successfully!")
    else:
        print("Student ID not found.")


# DELETE - Delete Student
def delete_student():
    student_id = int(input("Enter Student ID to delete: "))

    query = "DELETE FROM student WHERE id=%s"

    cursor.execute(query, (student_id,))
    con.commit()

    if cursor.rowcount > 0:
        print("Student deleted successfully!")
    else:
        print("Student ID not found.")


# Main Menu
while True:

    print("\n STUDENT DATABASE MANAGEMENT SYSTEM")
    print("1. Add Student")
    print("2. Display Students")
    print("3. Update Student")
    print("4. Delete Student")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        add_student()

    elif choice == "2":
        display_students()

    elif choice == "3":
        update_student()

    elif choice == "4":
        delete_student()

    elif choice == "5":
        print("Program terminated.")
        break
```

```
    else:
        print("Invalid choice!")


cursor.close()
con.close()
```

OUTPUT :-

```
 STUDENT DATABASE MANAGEMENT SYSTEM
1. Add Student
2. Display Students
3. Update Student
4. Delete Student
5. Exit
Enter your choice: 1
Enter Student ID: 102
Enter Student Name: XYZ
Enter Age: 20
Enter Course: Ai-ML
Enter Marks: 85
Student added successfully!
```

Practical 2: Library Database System

Description:
Create a library database and manage book records through Python.

Concepts Covered
- Database Connectivity
- SQL Queries

CODE :-

```sql
CREATE DATABASE library_db;
USE library_db;

CREATE TABLE books (
    book_id INT PRIMARY KEY,
    title VARCHAR(100),
    author VARCHAR(100),
    category VARCHAR(50),
    price FLOAT,
    quantity INT
);
```

```python
import mysql.connector

# Database connection
con = mysql.connector.connect(
    host="localhost",
    user="root",
    password="MySQL@12345",
    database="library_db"
)

cursor = con.cursor()


# Add Book
def add_book():
    book_id = int(input("Enter Book ID: "))
    title = input("Enter Book Title: ")
    author = input("Enter Author Name: ")
    category = input("Enter Category: ")
    price = float(input("Enter Price: "))
    quantity = int(input("Enter Quantity: "))

    query = """
```

```
    INSERT INTO books
    (book_id, title, author, category, price, quantity)
    VALUES (%s, %s, %s, %s, %s, %s)
    """

    values = (
        book_id,
        title,
        author,
        category,
        price,
        quantity
    )

    cursor.execute(query, values)
    con.commit()

    print("Book added successfully!")


# Display Books
def display_books():
    cursor.execute("SELECT * FROM books")

    records = cursor.fetchall()

    if len(records) == 0:
        print("No books found.")
    else:
        print("\n BOOK RECORDS ")

        for book in records:
            print(
                "ID:", book[0],
                "| Title:", book[1],
                "| Author:", book[2],
                "| Category:", book[3],
                "| Price:", book[4],
                "| Quantity:", book[5]
            )


# Search Book
def search_book():
    book_id = int(input("Enter Book ID to search: "))

    query = "SELECT * FROM books WHERE book_id = %s"

    cursor.execute(query, (book_id,))

    book = cursor.fetchone()

    if book:
        print("\nBook Found")
        print("Book ID:", book[0])
        print("Title:", book[1])
        print("Author:", book[2])
        print("Category:", book[3])
        print("Price:", book[4])
        print("Quantity:", book[5])
    else:
        print("Book not found.")


# Update Book
def update_book():
```

```
    book_id = int(input("Enter Book ID to update: "))

    title = input("Enter New Title: ")
    author = input("Enter New Author: ")
    category = input("Enter New Category: ")
    price = float(input("Enter New Price: "))
    quantity = int(input("Enter New Quantity: "))

    query = """
    UPDATE books
    SET title=%s,
        author=%s,
        category=%s,
        price=%s,
        quantity=%s
    WHERE book_id=%s
    """

    values = (
        title,
        author,
        category,
        price,
        quantity,
        book_id
    )

    cursor.execute(query, values)
    con.commit()

    if cursor.rowcount > 0:
        print("Book updated successfully!")
    else:
        print("Book ID not found.")


# Delete Book
def delete_book():
    book_id = int(input("Enter Book ID to delete: "))

    query = "DELETE FROM books WHERE book_id=%s"

    cursor.execute(query, (book_id,))
    con.commit()

    if cursor.rowcount > 0:
        print("Book deleted successfully!")
    else:
        print("Book ID not found.")


# Main Menu
while True:

    print("\n========== LIBRARY DATABASE SYSTEM ==========")
    print("1. Add Book")
    print("2. Display Books")
    print("3. Search Book")
    print("4. Update Book")
    print("5. Delete Book")
    print("6. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        add_book()
```

```
    elif choice == "2":
        display_books()

    elif choice == "3":
        search_book()

    elif choice == "4":
        update_book()

    elif choice == "5":
        delete_book()

    elif choice == "6":
        print("Program terminated.")
        break

    else:
        print("Invalid choice!")


cursor.close()
con.close()
```

OUTPUT :-

```
========== LIBRARY DATABASE SYSTEM ==========
1. Add Book
2. Display Books
3. Search Book
4. Update Book
5. Delete Book
6. Exit
Enter your choice: 1
Enter Book ID: 1
Enter Book Title: RICH DAD POOR DAD
Enter Author Name: ROBERT KIYOSAKI
Enter Category: MONEY FLOW
Enter Price: 200
Enter Quantity: 1
Book added successfully!

========== LIBRARY DATABASE SYSTEM ==========
1. Add Book
2. Display Books
3. Search Book
4. Update Book
5. Delete Book
6. Exit
Enter your choice: 2

 BOOK RECORDS
ID: 1 | Title: RICH DAD POOR DAD | Author: ROBERT KIYOSAKI | Category: MONEY FLOW | Price: 200.0 | Qua
ntity: 1
```

Practical 3: Employee Payroll Database

Description:
Store employee details in MySQL and generate payroll reports using Python.

Concepts Covered

- Insert
- Update
- Select

CODE :-

```sql
CREATE DATABASE payroll_db;

USE payroll_db;

CREATE TABLE employee (
    emp_id INT PRIMARY KEY,
    name VARCHAR(100),
    department VARCHAR(50),
    basic_salary FLOAT,
    allowance FLOAT,
    deduction FLOAT
);

DESC employee;
```

3:12

Result Grid Filter Rows: Search

Field	Type	Null	Key	Default
emp_id	int	NO	PRI	NULL
name	varchar(100)	YES		NULL
department	varchar(50)	YES		NULL
basic_salary	float	YES		NULL
allowance	float	YES		NULL
deduction	float	YES		NULL

```python
import mysql.connector

# Connect to MySQL
con = mysql.connector.connect(
    host="localhost",
    user="root",
    password="MySQL@12345",
    database="payroll_db"
)

cursor = con.cursor()
```

```python
# INSERT - Add Employee
def add_employee():
    emp_id = int(input("Enter Employee ID: "))
    name = input("Enter Employee Name: ")
    department = input("Enter Department: ")
    basic_salary = float(input("Enter Basic Salary: "))
    allowance = float(input("Enter Allowance: "))
    deduction = float(input("Enter Deduction: "))

    query = """
    INSERT INTO employee
    (emp_id, name, department, basic_salary, allowance, deduction)
    VALUES (%s, %s, %s, %s, %s, %s)
    """

    values = (
        emp_id,
        name,
        department,
        basic_salary,
        allowance,
        deduction
    )

    cursor.execute(query, values)
    con.commit()

    print("Employee added successfully!")


# UPDATE - Update Employee
def update_employee():
    emp_id = int(input("Enter Employee ID to update: "))

    basic_salary = float(input("Enter New Basic Salary: "))
    allowance = float(input("Enter New Allowance: "))
    deduction = float(input("Enter New Deduction: "))

    query = """
    UPDATE employee
    SET basic_salary=%s,
        allowance=%s,
        deduction=%s
    WHERE emp_id=%s
    """

    values = (
        basic_salary,
        allowance,
        deduction,
        emp_id
    )

    cursor.execute(query, values)
    con.commit()

    if cursor.rowcount > 0:
        print("Employee updated successfully!")
    else:
        print("Employee ID not found.")


# SELECT - Generate Payroll Report
def payroll_report():
    cursor.execute("SELECT * FROM employee")
```

```
    employees = cursor.fetchall()

    if len(employees) == 0:
        print("No employee records found.")
        return

    print("\n EMPLOYEE PAYROLL REPORT ")

    for emp in employees:

        emp_id = emp[0]
        name = emp[1]
        department = emp[2]
        basic_salary = emp[3]
        allowance = emp[4]
        deduction = emp[5]

        # Calculate gross salary
        gross_salary = basic_salary + allowance

        # Calculate net salary
        net_salary = gross_salary - deduction

        print("\nEmployee ID:", emp_id)
        print("Name:", name)
        print("Department:", department)
        print("Basic Salary:", basic_salary)
        print("Allowance:", allowance)
        print("Deduction:", deduction)
        print("Gross Salary:", gross_salary)
        print("Net Salary:", net_salary)


# Main Menu
while True:

    print("\n========== EMPLOYEE PAYROLL SYSTEM ==========")
    print("1. Add Employee")
    print("2. Update Employee")
    print("3. Generate Payroll Report")
    print("4. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        add_employee()

    elif choice == "2":
        update_employee()

    elif choice == "3":
        payroll_report()

    elif choice == "4":
        print("Program terminated.")
        break

    else:
        print("Invalid choice!")


cursor.close()
con.close()
```

OUTPUT :-

```
========== EMPLOYEE PAYROLL SYSTEM ==========
1. Add Employee
2. Update Employee
3. Generate Payroll Report
4. Exit
Enter your choice: 1
Enter Employee ID: 101
Enter Employee Name: XYZ
Enter Department: IT
Enter Basic Salary: 50000
Enter Allowance: 20000
Enter Deduction: 5000
Employee added successfully!

========== EMPLOYEE PAYROLL SYSTEM ==========
1. Add Employee
2. Update Employee
3. Generate Payroll Report
4. Exit
Enter your choice: 3

 EMPLOYEE PAYROLL REPORT

Employee ID: 101
Name: XYZ
Department: IT
Basic Salary: 50000.0
Allowance: 20000.0
Deduction: 5000.0
Gross Salary: 70000.0
Net Salary: 65000.0
```

Practical 4: Hospital Patient Record Management

Description:
Maintain patient records in MySQL with options for registration, searching, updating, and deleting records.

Concepts Covered
● Database Programming

CODE :-

```sql
CREATE DATABASE hospital_db;

USE hospital_db;

CREATE TABLE patient (
    patient_id INT PRIMARY KEY,
    name VARCHAR(100),
    age INT,
    gender VARCHAR(10),
    phone VARCHAR(15),
    disease VARCHAR(100),
    doctor VARCHAR(100)
);

DESC patient;
```

3:13

esult Grid Filter Rows: Search

Field	Type	Null	Key	Default	Extra
patient_id	int	NO	PRI	NULL	
name	varchar(100)	YES		NULL	
age	int	YES		NULL	
gender	varchar(10)	YES		NULL	
phone	varchar(15)	YES		NULL	
disease	varchar(100)	YES		NULL	
doctor	varchar(100)	YES		NULL	

```python
import mysql.connector

# Connect to MySQL
con = mysql.connector.connect(
    host="localhost",
    user="root",
    password="MySQL@12345",
    database="hospital_db"
)

cursor = con.cursor()

# Register Patient
def register_patient():

    patient_id = int(input("Enter Patient ID: "))
    name = input("Enter Patient Name: ")
    age = int(input("Enter Age: "))
    gender = input("Enter Gender: ")
    phone = input("Enter Phone Number: ")
    disease = input("Enter Disease: ")
    doctor = input("Enter Doctor Name: ")
```

```
    query = """
    INSERT INTO patient
    (patient_id, name, age, gender, phone, disease, doctor)
    VALUES (%s, %s, %s, %s, %s, %s, %s)
    """

    values = (
        patient_id,
        name,
        age,
        gender,
        phone,
        disease,
        doctor
    )

    cursor.execute(query, values)
    con.commit()

    print("Patient registered successfully!")


# Display All Patients
def display_patients():

    cursor.execute("SELECT * FROM patient")

    patients = cursor.fetchall()

    if len(patients) == 0:
        print("No patient records found.")
        return

    print("\n PATIENT RECORDS ")

    for patient in patients:

        print("\nPatient ID:", patient[0])
        print("Name:", patient[1])
        print("Age:", patient[2])
        print("Gender:", patient[3])
        print("Phone:", patient[4])
        print("Disease:", patient[5])
        print("Doctor:", patient[6])


# Search Patient
def search_patient():

    patient_id = int(input("Enter Patient ID to search: "))

    query = "SELECT * FROM patient WHERE patient_id = %s"

    cursor.execute(query, (patient_id,))

    patient = cursor.fetchone()

    if patient:

        print("\nPATIENT FOUND ")
        print("Patient ID:", patient[0])
        print("Name:", patient[1])
        print("Age:", patient[2])
        print("Gender:", patient[3])
        print("Phone:", patient[4])
```

```
        print("Disease:", patient[5])
        print("Doctor:", patient[6])

    else:
        print("Patient not found.")


# Update Patient
def update_patient():

    patient_id = int(input("Enter Patient ID to update: "))

    name = input("Enter New Name: ")
    age = int(input("Enter New Age: "))
    gender = input("Enter New Gender: ")
    phone = input("Enter New Phone Number: ")
    disease = input("Enter New Disease: ")
    doctor = input("Enter New Doctor Name: ")

    query = """
    UPDATE patient
    SET name=%s,
        age=%s,
        gender=%s,
        phone=%s,
        disease=%s,
        doctor=%s
    WHERE patient_id=%s
    """

    values = (
        name,
        age,
        gender,
        phone,
        disease,
        doctor,
        patient_id
    )

    cursor.execute(query, values)
    con.commit()

    if cursor.rowcount > 0:
        print("Patient record updated successfully!")
    else:
        print("Patient ID not found.")


# Delete Patient
def delete_patient():

    patient_id = int(input("Enter Patient ID to delete: "))

    query = "DELETE FROM patient WHERE patient_id = %s"

    cursor.execute(query, (patient_id,))
    con.commit()

    if cursor.rowcount > 0:
        print("Patient record deleted successfully!")
    else:
        print("Patient ID not found.")


# Main Menu
```

```
while True:

    print("\nHOSPITAL PATIENT MANAGEMENT ")
    print("1. Register Patient")
    print("2. Display Patients")
    print("3. Search Patient")
    print("4. Update Patient")
    print("5. Delete Patient")
    print("6. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        register_patient()

    elif choice == "2":
        display_patients()

    elif choice == "3":
        search_patient()

    elif choice == "4":
        update_patient()

    elif choice == "5":
        delete_patient()

    elif choice == "6":
        print("Program terminated.")
        break

    else:
        print("Invalid choice!")


cursor.close()
con.close()
```

OUTPUT :-

```
HOSPITAL PATIENT MANAGEMENT
1. Register Patient
2. Display Patients
3. Search Patient
4. Update Patient
5. Delete Patient
6. Exit
Enter your choice: 1
Enter Patient ID: 101
Enter Patient Name: XYZ
Enter Age: 20
Enter Gender: Male
Enter Phone Number: 9998887777
Enter Disease: CORONA
Enter Doctor Name: XYZ
Patient registered successfully!
```

Practical 5: Online Food Ordering Database

Description:
Develop a Python application that manages customers, menu items, and orders using a MySQL database.

Concepts Covered
- Database Connectivity
- Transactions

CODE :-

```sql
CREATE DATABASE food_order_db;
USE food_order_db;

CREATE TABLE customer (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100),
    phone VARCHAR(15),
    address VARCHAR(200)
);

CREATE TABLE menu (
    item_id INT PRIMARY KEY,
    item_name VARCHAR(100),
    category VARCHAR(50),
    price FLOAT,
    available INT
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    item_id INT,
    quantity INT,
    total_amount FLOAT,
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id),
    FOREIGN KEY (item_id) REFERENCES menu(item_id)
);

SHOW TABLES;
```

18:23

esult Grid | Filter Rows: Search | Export:

Tables_in_food_order_...
customer
menu
orders

```python
import mysql.connector

# Connect to MySQL
con = mysql.connector.connect(
    host="localhost",
    user="root",
    password="MySQL@12345",
    database="food_order_db"
)

cursor = con.cursor()


# Add Customer
def add_customer():

    customer_id = int(input("Enter Customer ID: "))
    name = input("Enter Customer Name: ")
    phone = input("Enter Phone Number: ")
    address = input("Enter Address: ")
```

```
    query = """
    INSERT INTO customer
    (customer_id, name, phone, address)
    VALUES (%s, %s, %s, %s)
    """

    values = (customer_id, name, phone, address)

    cursor.execute(query, values)
    con.commit()

    print("Customer added successfully!")


# Add Menu Item
def add_menu_item():

    item_id = int(input("Enter Item ID: "))
    item_name = input("Enter Item Name: ")
    category = input("Enter Category: ")
    price = float(input("Enter Price: "))

    query = """
    INSERT INTO menu
    (item_id, item_name, category, price, available)
    VALUES (%s, %s, %s, %s, %s)
    """

    values = (item_id, item_name, category, price, 1)

    cursor.execute(query, values)
    con.commit()

    print("Menu item added successfully!")


# Display Menu
def display_menu():

    cursor.execute("SELECT * FROM menu")

    items = cursor.fetchall()

    if len(items) == 0:
        print("No menu items found.")
        return

    print("\n MENU ")

    for item in items:

        status = "Available" if item[4] == 1 else "Not Available"

        print(
            "ID:", item[0],
            "| Item:", item[1],
            "| Category:", item[2],
            "| Price:", item[3],
            "|", status
        )


# Place Order
def place_order():
```

```
    try:

        order_id = int(input("Enter Order ID: "))
        customer_id = int(input("Enter Customer ID: "))
        item_id = int(input("Enter Item ID: "))
        quantity = int(input("Enter Quantity: "))

        # Check customer
        cursor.execute(
            "SELECT * FROM customer WHERE customer_id = %s",
            (customer_id,)
        )

        customer = cursor.fetchone()

        if customer is None:
            print("Customer not found.")
            return

        # Check menu item
        cursor.execute(
            "SELECT price, available FROM menu WHERE item_id = %s",
            (item_id,)
        )

        item = cursor.fetchone()

        if item is None:
            print("Menu item not found.")
            return

        price = item[0]
        available = item[1]

        if available == 0:
            print("Item is currently unavailable.")
            return

        # Calculate total
        total_amount = price * quantity

        # Insert order
        query = """
        INSERT INTO orders
        (order_id, customer_id, item_id, quantity, total_amount)
        VALUES (%s, %s, %s, %s, %s)
        """

        values = (
            order_id,
            customer_id,
            item_id,
            quantity,
            total_amount
        )

        cursor.execute(query, values)

        # Commit transaction
        con.commit()

        print("\nOrder placed successfully!")
        print("Total Amount:", total_amount)

    except Exception as e:
```

```
        # Rollback transaction
        con.rollback()

        print("Order failed.")
        print("Transaction rolled back.")
        print("Error:", e)


# Display Orders
def display_orders():

    query = """
    SELECT
        orders.order_id,
        customer.name,
        menu.item_name,
        orders.quantity,
        orders.total_amount
    FROM orders
    JOIN customer
        ON orders.customer_id = customer.customer_id
    JOIN menu
        ON orders.item_id = menu.item_id
    """

    cursor.execute(query)

    orders = cursor.fetchall()

    if len(orders) == 0:
        print("No orders found.")
        return

    print("\nORDERS ")

    for order in orders:

        print(
            "Order ID:", order[0],
            "| Customer:", order[1],
            "| Item:", order[2],
            "| Quantity:", order[3],
            "| Total:", order[4]
        )


# Main Menu
while True:

    print("\n ONLINE FOOD ORDERING SYSTEM ")
    print("1. Add Customer")
    print("2. Add Menu Item")
    print("3. Display Menu")
    print("4. Place Order")
    print("5. Display Orders")
    print("6. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        add_customer()

    elif choice == "2":
        add_menu_item()

    elif choice == "3":
```

```
        display_menu()

    elif choice == "4":
        place_order()

    elif choice == "5":
        display_orders()

    elif choice == "6":
        print("Program terminated.")
        break

    else:
        print("Invalid choice!")


cursor.close()
con.close()
```

OUTPUT :-

```
 ONLINE FOOD ORDERING SYSTEM
1. Add Customer
2. Add Menu Item
3. Display Menu
4. Place Order
5. Display Orders
6. Exit
Enter your choice: 2
Enter Item ID: 101
Enter Item Name: Pizza
Enter Category: Veg
Enter Price: 1000
Menu item added successfully!

 ONLINE FOOD ORDERING SYSTEM
1. Add Customer
2. Add Menu Item
3. Display Menu
4. Place Order
5. Display Orders
6. Exit
Enter your choice: 3

 MENU
ID: 101 | Item: Pizza | Category: Veg | Price: 1000.0 | Available
```

Practical 6: College Examination Management System

Description:
Create a database-driven application for storing examination results, retrieving student marks,
generating grade reports, and updating examination records.

Concepts Covered
- Python-MySQL Connectivity
- CRUD Operations
- Database Tables

CODE :-

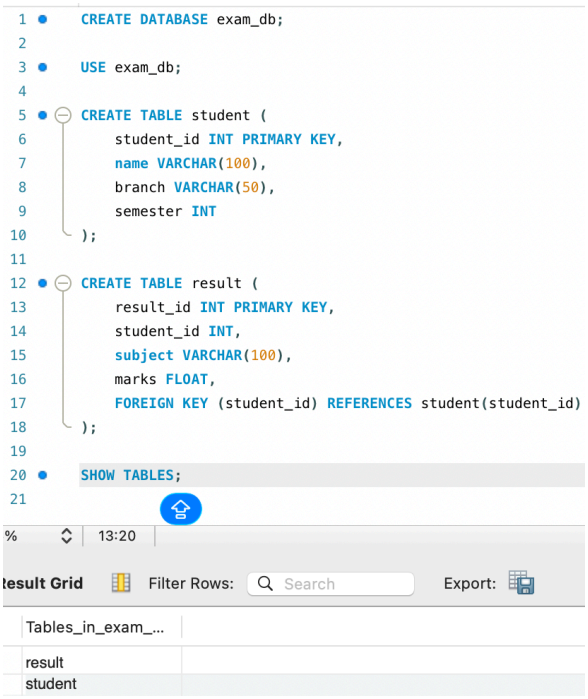
```sql
CREATE DATABASE exam_db;

USE exam_db;

CREATE TABLE student (
    student_id INT PRIMARY KEY,
    name VARCHAR(100),
    branch VARCHAR(50),
    semester INT
);

CREATE TABLE result (
    result_id INT PRIMARY KEY,
    student_id INT,
    subject VARCHAR(100),
    marks FLOAT,
    FOREIGN KEY (student_id) REFERENCES student(student_id)
);

SHOW TABLES;
```




```python
import mysql.connector

# Connect to MySQL
con = mysql.connector.connect(
    host="localhost",
    user="root",
    password="MySQL@12345",
    database="exam_db"
)

cursor = con.cursor()

# Add Student
def add_student():

    student_id = int(input("Enter Student ID: "))
```

```
    name = input("Enter Student Name: ")
    branch = input("Enter Branch: ")
    semester = int(input("Enter Semester: "))

    query = """
    INSERT INTO student
    (student_id, name, branch, semester)
    VALUES (%s, %s, %s, %s)
    """

    values = (student_id, name, branch, semester)

    cursor.execute(query, values)
    con.commit()

    print("Student added successfully!")


# Add Examination Result
def add_result():

    result_id = int(input("Enter Result ID: "))
    student_id = int(input("Enter Student ID: "))
    subject = input("Enter Subject: ")
    marks = float(input("Enter Marks: "))

    query = """
    INSERT INTO result
    (result_id, student_id, subject, marks)
    VALUES (%s, %s, %s, %s)
    """

    values = (
        result_id,
        student_id,
        subject,
        marks
    )

    cursor.execute(query, values)
    con.commit()

    print("Result added successfully!")


# Display Student Marks
def display_marks():

    student_id = int(input("Enter Student ID: "))

    query = """
    SELECT
        student.student_id,
        student.name,
        student.branch,
        student.semester,
        result.subject,
        result.marks
    FROM student
    JOIN result
        ON student.student_id = result.student_id
    WHERE student.student_id = %s
    """

    cursor.execute(query, (student_id,))
```

```
    records = cursor.fetchall()

    if len(records) == 0:
        print("No examination records found.")
        return

    print("\n========== STUDENT MARKS ==========")

    for record in records:

        print(
            "ID:", record[0],
            "| Name:", record[1],
            "| Branch:", record[2],
            "| Semester:", record[3],
            "| Subject:", record[4],
            "| Marks:", record[5]
        )


# Generate Grade
def generate_grade(marks):

    if marks >= 90:
        return "A+"
    elif marks >= 80:
        return "A"
    elif marks >= 70:
        return "B+"
    elif marks >= 60:
        return "B"
    elif marks >= 50:
        return "C"
    elif marks >= 40:
        return "D"
    else:
        return "F"


# Generate Grade Report
def grade_report():

    student_id = int(input("Enter Student ID: "))

    query = """
    SELECT
        student.name,
        student.branch,
        student.semester,
        result.subject,
        result.marks
    FROM student
    JOIN result
        ON student.student_id = result.student_id
    WHERE student.student_id = %s
    """

    cursor.execute(query, (student_id,))

    records = cursor.fetchall()

    if len(records) == 0:
        print("No records found.")
        return

    total_marks = 0
```

```
    print("\nGRADE REPORT ")

    print("Student Name:", records[0][0])
    print("Branch:", records[0][1])
    print("Semester:", records[0][2])

    print("\nSubject-wise Result:")

    for record in records:

        subject = record[3]
        marks = record[4]

        grade = generate_grade(marks)

        print(
            "Subject:", subject,
            "| Marks:", marks,
            "| Grade:", grade
        )

        total_marks += marks

    number_of_subjects = len(records)

    percentage = total_marks / number_of_subjects

    print("\nTotal Marks:", total_marks)
    print("Percentage:", percentage, "%")

    if percentage >= 40:
        print("Result: PASS")
    else:
        print("Result: FAIL")


# Update Result
def update_result():

    result_id = int(input("Enter Result ID to update: "))

    marks = float(input("Enter New Marks: "))

    query = """
    UPDATE result
    SET marks=%s
    WHERE result_id=%s
    """

    values = (marks, result_id)

    cursor.execute(query, values)
    con.commit()

    if cursor.rowcount > 0:
        print("Examination record updated successfully!")
    else:
        print("Result ID not found.")


# Delete Result
def delete_result():

    result_id = int(input("Enter Result ID to delete: "))
```

```
    query = "DELETE FROM result WHERE result_id=%s"

    cursor.execute(query, (result_id,))
    con.commit()

    if cursor.rowcount > 0:
        print("Result deleted successfully!")
    else:
        print("Result ID not found.")


# Main Menu
while True:

    print("\nCOLLEGE EXAMINATION MANAGEMENT SYSTEM ")

    print("1. Add Student")
    print("2. Add Examination Result")
    print("3. Display Student Marks")
    print("4. Generate Grade Report")
    print("5. Update Examination Record")
    print("6. Delete Examination Record")
    print("7. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        add_student()

    elif choice == "2":
        add_result()

    elif choice == "3":
        display_marks()

    elif choice == "4":
        grade_report()

    elif choice == "5":
        update_result()

    elif choice == "6":
        delete_result()

    elif choice == "7":
        print("Program terminated.")
        break

    else:
        print("Invalid choice!")


cursor.close()
con.close()
```

OUTPUT :-

```
COLLEGE EXAMINATION MANAGEMENT SYSTEM
1. Add Student
2. Add Examination Result
3. Display Student Marks
4. Generate Grade Report
5. Update Examination Record
6. Delete Examination Record
7. Exit
Enter your choice: 1
Enter Student ID: 101
Enter Student Name: XYZ
Enter Branch: AI-ML
Enter Semester: 5
Student added successfully!

COLLEGE EXAMINATION MANAGEMENT SYSTEM
1. Add Student
2. Add Examination Result
3. Display Student Marks
4. Generate Grade Report
5. Update Examination Record
6. Delete Examination Record
7. Exit
Enter your choice: 3
Enter Student ID: 101
No examination records found.
```